

**PANEVROPSKI UNIVERZITET APEIRON, BANJA LUKA
FAKULTET INFORMACIONIH TEHNOLOGIJA**

Redovne studije

Smjer : “Inženjering Informacionih Tehnologija“

Predmet:

Konkurentno računarstvo – paralelno programiranje

**ANALIZA PERFORMANSI SEKVENCIJALNOG I
PARALELNOG IZVRŠAVANJA PROGRAMA U
PYTHONU
(Seminarski rad)**

Predmetni nastavnik

Prof.dr Negovan Stamenković

Student: Nikolina Bekić

Br. Indeksa : 21-22-FIT-IIT1-PRO-240

Banja Luka, septembar 2026

SADRŽAJ

UVOD	1
1. OSNOVNI KONCEPTI IZVRŠAVANJA PROGRAMA	2
1.1. Sekvencijalno izvršavanje programa	2
1.2. Razvoj Pythona i njegova primjena	2
1.3. Problem performansi kod velikih skupova podataka	2
1.4. Uloga višejezgrenih procesora u savremenom računarstvu	3
2. KONKURENTNOST I PARALELIZAM U PYTHONU	4
2.1. Razlika između konkurentnosti i paralelizma	4
2.2. Procesi kao nezavisne jedinice izvršavanja	4
2.3. Niti i njihova primjena	4
2.4. Global Interpreter Lock i njegov uticaj na niti	4
2.5. Python biblioteke za paralelno izvršavanje	5
2.5.1. Modul threading	5
2.5.2. Modul multiprocessing	6
2.5.3. Modul concurrent.futures	7
3. PRAKTIČNA KOMPARACIJA SEKVENCIJALNOG I PARALELNOG KODA	8
3.1. Primjer 1: Suma kvadrata brojeva	8
3.2. Primjer 2: Obrada liste ocjena	10
3.3. Primjer 3: Provjera djeljivosti velikog broja elemenata	12
3.4. Primjer 4: Simulacija više sporih operacija	14
3.5. Primjer 5: Analiza više tekstualnih dokumenata	15
3.6. Primjer 6: Računanje prostih brojeva	17
3.7. Primjer 7: Paralelna obrada redova matrice	19
3.8. Primjer 8: Filtriranje podataka iz velike liste	20
3.9. Primjer 9: Generisanje SHA-256 hash vrijednosti	22
3.10. Primjer 10: Simulacija senzorskih očitavanja	24
3.11. Primjer 11: Pretvaranje velikog broja tekstova u velika slova	25
3.12. Primjer 12: Traženje maksimalne vrijednosti u dijelovima liste	27
3.13. Primjer 13: Simulacija Monte Carlo metode	28
3.14. Primjer 14: Sortiranje nezavisnih nizova	31
3.15. Primjer 15: Obrada jednostavne slike kao matrice	33
4. PREGLED REZULTATA	35
ZAKLJUČAK	37

POPIS SLIKA.....	38
LITERATURA	39

UVOD

Performanse programa predstavljaju važan aspekt razvoja softvera, posebno kada se radi sa velikim količinama podataka ili zadacima koji zahtijevaju veliki broj računskih operacija. Sa razvojem višejezgrenih procesora pojavila se mogućnost istovremenog izvršavanja više dijelova programa, čime se u odgovarajućim slučajevima može značajno smanjiti ukupno vrijeme izvršavanja. Međutim, samo korištenje više procesorskih jezgara ne garantuje bolje performanse, jer paralelno izvršavanje uvodi dodatne troškove organizacije, komunikacije i prenosa podataka.

Python je programski jezik koji se široko koristi za razvoj različitih vrsta aplikacija, uključujući obradu podataka, automatizaciju, naučna istraživanja i razvoj softvera. Za konkurentno i paralelno izvršavanje Python pruža više mehanizama, među kojima se posebno izdvajaju niti i procesi, kao i biblioteka `concurrent.futures`, koja omogućava jednostavnije upravljanje ovim oblicima izvršavanja. Izbor odgovarajućeg pristupa zavisi od prirode zadatka, odnosno od toga da li je operacija više opterećena računanjem ili čekanjem na ulazno-izlazne operacije.

Cilj ovog rada je analiza performansi sekvencijalnog i paralelnog izvršavanja programa u Pythonu kroz veći broj različitih praktičnih zadataka. U radu je obrađeno 15 primjera koji obuhvataju matematičke proračune, obradu velikih lista, rad sa tekstom, provjeru prostih brojeva, obradu matrica, filtriranje podataka, generisanje hash vrijednosti, simulaciju senzorskih očitavanja, Monte Carlo simulaciju, sortiranje nizova i jednostavnu obradu slike.

Za svaki primjer implementirane su sekvencijalna i paralelna verzija programa, nakon čega je izmjereno vrijeme njihovog izvršavanja i izračunato ostvareno ubrzanje. Na osnovu dobijenih rezultata analizirano je u kojim situacijama paralelno izvršavanje donosi stvarno poboljšanje performansi, a kada dodatni troškovi paralelizacije dovode do sporijeg izvršavanja u odnosu na jednostavniji sekvencijalni pristup.

Na taj način rad nastoji pokazati da izbor između sekvencijalnog i paralelnog izvršavanja ne treba zasnivati samo na broju dostupnih procesorskih jezgara, već prvenstveno na karakteristikama konkretnog problema, količini podataka, složenosti operacija i troškovima komunikacije između niti ili procesa.

1. OSNOVNI KONCEPTI IZVRŠAVANJA PROGRAMA

1.1. Sekvencijalno izvršavanje programa

Sekvencijalno izvršavanje predstavlja osnovni i najjednostavniji način rada programa. Kod ovog pristupa program izvršava naredbe redom kojim su napisane. Kada se jedna naredba završi, prelazi se na sljedeću. Ovakav način izvršavanja je jednostavan za razumijevanje jer odgovara prirodnom načinu razmišljanja.

Na primjer, ako program treba da pročita listu brojeva, izračuna njihov zbir i zatim prikaže rezultat, sve se odvija u jednom toku. Prvo se učitavaju podaci, zatim se obrađuju, a zatim prikazuje rezultat. Programer lako može pratiti šta se dešava u svakom trenutku.

Prednost sekvencijalnog programiranja je njegova jednostavnost. Kod je obično kraći, pregledniji i lakše ga je testirati. Greške se lakše pronalaze jer postoji jedan glavni tok izvršavanja. Zbog toga se sekvencijalni pristup i dalje koristi u velikom broju programa.

Ipak, sekvencijalno izvršavanje ima ograničenja. Ako se isti zadatak mora izvršiti nad velikim brojem elemenata, program može trajati dugo. Takođe, ako računar ima više procesorskih jezgara, sekvencijalni program najčešće neće iskoristiti puni potencijal hardvera.

1.2. Razvoj Pythona i njegova primjena

Python je programski jezik visokog nivoa koji je kreirao Guido van Rossum. Prema zvaničnoj dokumentaciji, Python je nastao početkom devedesetih godina u CWI institutu u Holandiji kao nasljednik programskog jezika ABC.

Danas je Python jedan od najpopularnijih programskih jezika. Koristi se u obrazovanju, web razvoju, automatizaciji, administraciji sistema, analizi podataka, vještačkoj inteligenciji, mašinskom učenju i naučnim istraživanjima. Njegova sintaksa je jednostavna i čitljiva, pa omogućava programerima da se više fokusiraju na rješavanje problema, a manje na tehničke detalje jezika.

Python ima veliki broj standardnih i dodatnih biblioteka. Za temu ovog rada posebno su važne biblioteke koje omogućavaju konkurentno i paralelno izvršavanje, kao što su `threading`, `multiprocessing` i `concurrent.futures`.

1.3. Problem performansi kod velikih skupova podataka

Performanse programa zavise od mnogo faktora. Neki programi su spori zbog lošeg algoritma, neki zbog velike količine podataka, a neki zbog toga što ne koriste dostupne hardverske resurse. Kada program obrađuje mali broj elemenata, razlika između sekvencijalnog i paralelnog izvršavanja često nije značajna. Međutim, kada se broj elemenata poveća, razlika može postati veoma vidljiva.

Tipični problemi kod kojih sekvencijalno izvršavanje može biti sporo su obrada velikih lista, pretraga podataka, obrada više fajlova, obrada slika, simulacije i matematički proračuni. Ako se svaki dio posla može izvršavati nezavisno, paralelizam može biti dobro rješenje.

Važno je naglasiti da paralelizam ne znači automatski ubrzanje. Paralelno izvršavanje uvodi dodatni trošak, kao što je kreiranje niti ili procesa, prenos podataka i sinhronizacija. Zbog toga je neophodno mjeriti vrijeme izvršavanja i uporediti rezultate.

1.4. Uloga višejezgrenih procesora u savremenom računarstvu

Savremeni računari najčešće imaju procesore sa više jezgara. Svako jezgro može izvršavati dio posla, pa se pravilno napisan program može izvršavati brže. Međutim, hardver sam po sebi nije dovoljan. Program mora biti organizovan tako da može koristiti više jezgara.

Ako je program napisan isključivo sekvencijalno, on će uglavnom izvršavati jedan tok instrukcija. To znači da se potencijal višejezgrenog procesora ne koristi u potpunosti. Paralelno programiranje omogućava da se zadatak podijeli, a zatim izvršava istovremeno na više jezgara.

U Pythonu izbor između niti i procesa zavisi od prirode zadatka. Ako zadatak najviše vremena provodi čekajući, na primjer kod fajlova ili mreže, niti mogu biti dobar izbor. Ako je zadatak računski zahtjevan, procesi su često bolji izbor jer mogu zaobići ograničenja Global Interpreter Lock mehanizma.

2. KONKURENTNOST I PARALELIZAM U PYTHONU

2.1. Razlika između konkurentnosti i paralelizma

„Konkurentnost i paralelizam su povezani pojmovi, ali ne znače isto. Konkurentnost znači da program može upravljati sa više zadataka u istom vremenskom periodu. Ti zadaci se mogu smjenjivati, ali ne moraju nužno biti izvršeni u istom trenutku.

Paralelizam znači da se više zadataka zaista izvršava istovremeno. Za pravi paralelizam obično je potrebno više procesorskih jezgara. Ako program ima četiri nezavisna zadatka, a računar četiri jezgra, moguće je da se svaki zadatak izvršava na posebnom jezgru. Python podržava oba pristupa. Konkurentnost se često postiže nitima, dok se paralelizam za računski zahtjevne zadatke obično postiže procesima“ [1].

2.2. Proces i kao nezavisne jedinice izvršavanja

„Proces je nezavisna jedinica izvršavanja programa. Svaki proces ima svoju memoriju i svoje resurse. Zbog toga procesi ne dijele podatke direktno kao niti. Ova izolacija povećava stabilnost, ali komunikaciju između procesa čini složenijom.

U Pythonu se procesi koriste preko modula multiprocessing. Zvanična dokumentacija navodi da multiprocessing podržava kreiranje procesa i omogućava iskorišćavanje više procesora korišćenjem podprocesa umjesto niti“ [2].

Procesi su posebno korisni kod CPU-intenzivnih zadataka. Primjeri su računanje prostih brojeva, matematičke simulacije, obrada slika i veći numerički proračuni.

2.3. Niti i njihova primjena

„Nit je lakša jedinica izvršavanja unutar procesa. Više niti u jednom procesu dijeli memoriju tog procesa. Zbog toga su niti pogodne kada zadaci treba da komuniciraju ili koriste zajedničke podatke“ [3]. Niti su posebno korisne kod I/O zadataka. To su zadaci u kojima program čeka disk, mrežu, korisnički unos ili neki drugi spoljašnji resurs. Dok jedna nit čeka, druga može izvršavati drugi dio posla.

Međutim, rad sa nitima zahtijeva oprez. Ako više niti istovremeno pristupa istim podacima, mogu nastati greške. Zbog toga se često koriste mehanizmi zaključavanja, kao što su Lock, Semaphore i slični alati.

2.4. Global Interpreter Lock i njegov uticaj na niti

„Jedna od specifičnosti standardne CPython implementacije jeste Global Interpreter Lock, skraćeno GIL. GIL je mehanizam koji ograničava istovremeno izvršavanje Python bajtkoda u više niti. Zvanična dokumentacija objašnjava da nit mora držati GIL da bi radila sa Python objektima“ [4].

Zbog GIL-a, Python niti nisu idealne za računski zahtjevne zadatke. Na primjer, ako dvije niti pokušavaju izvršavati teške matematičke operacije, ne mora se dobiti očekivano ubrzanje. Međutim, kod I/O operacija GIL nije toliko veliki problem, jer se tokom čekanja može omogućiti izvršavanje druge niti. Zbog toga se u Pythonu često pravi razlika između CPU-bound i I/O-bound zadataka. Za CPU-bound zadatke bolji je multiprocessing, dok su za I/O-bound zadatke često dovoljne niti.

2.5. Python biblioteke za paralelno izvršavanje

„Python standardna biblioteka pruža više različitih mehanizama za konkurentno i paralelno izvršavanje programa. Njihova primjena zavisi od karakteristika zadatka, količine podataka, načina na koji se podaci obrađuju i vremena koje program provodi u računanju ili čekanju na spoljne resurse. Najčešće korišteni moduli za ovu namjenu su threading, multiprocessing i concurrent.futures.

Izbor odgovarajućeg mehanizma predstavlja važan dio optimizacije programa. Nije dovoljno samo pokrenuti više niti ili procesa i očekivati kraće vrijeme izvršavanja. Paralelno izvršavanje podrazumijeva dodatne troškove koji nastaju prilikom kreiranja niti ili procesa, raspoređivanja zadataka, komunikacije, sinhronizacije i prenosa podataka“ [1]. Kod malih i jednostavnih zadataka ovi troškovi mogu biti veći od koristi koju donosi paralelno izvršavanje. Zbog toga je potrebno analizirati samu prirodu zadatka prije izbora odgovarajućeg pristupa.

Kod zadataka koji većinu vremena provode čekajući na ulazno-izlazne operacije, kao što su mrežni zahtjevi, čitanje fajlova, komunikacija sa bazom podataka ili čekanje odgovora spoljnog uređaja, niti mogu biti veoma efikasne. Dok jedna nit čeka, druga može nastaviti sa izvršavanjem. Sa druge strane, kod zadataka koji zahtijevaju veliki broj računskih operacija, procesi mogu biti pogodniji jer omogućavaju korišćenje više procesorskih jezgara.

2.5.1. Modul threading

„Modul threading predstavlja standardni Python mehanizam za rad sa nitima. Nit predstavlja zasebnu jedinicu izvršavanja unutar procesa, pri čemu niti istog procesa dijele njegov memorijski prostor i druge resurse. Zahvaljujući tome, komunikacija između niti može biti jednostavnija nego komunikacija između odvojenih procesa.

Osnovna prednost niti jeste relativno mali trošak njihovog kreiranja i upravljanja u poređenju sa procesima. Nije potrebno kreirati zaseban Python interpreter i potpuno odvojen memorijski prostor za svaku nit. Zajednička memorija takođe omogućava jednostavno korišćenje podataka koje koristi više niti. Međutim, upravo dijeljenje podataka predstavlja i potencijalni problem jer više niti može istovremeno pristupati istim objektima“ [3]. Kod konkurentnog rada sa nitima može doći do situacija u kojima dvije ili više niti pokušavaju istovremeno promijeniti isti podatak. Takve situacije mogu dovesti do nekonzistentnih rezultata i predstavljaju primjer problema poznatog kao race condition. Zbog toga modul threading pruža različite mehanizme za sinhronizaciju, među kojima se nalaze Lock, RLock, Semaphore, Event i Condition.

Na primjer, Lock omogućava da samo jedna nit u određenom trenutku pristupa dijelu programa koji zahtijeva međusobno isključivanje. Na taj način moguće je zaštititi zajedničke podatke od istovremenih izmjena. Semaphore se može koristiti kada je potrebno ograničiti broj niti koje istovremeno pristupaju određenom resursu. Ovi mehanizmi povećavaju sigurnost konkurentnog programa, ali njihovo pretjerano korišćenje može smanjiti prednosti konkurentnog izvršavanja.

threading je posebno pogodan za I/O-bound zadatke. Ako program izvršava više mrežnih zahtjeva, čita podatke sa različitih uređaja ili čeka određeni vremenski period, više niti može omogućiti da se vrijeme čekanja jedne operacije iskoristi za izvršavanje druge. Zbog toga se niti često koriste u mrežnim aplikacijama, serverima, automatizaciji, radu sa spoljnim uređajima i sličnim sistemima.

Važno ograničenje predstavlja GIL, odnosno Global Interpreter Lock, u standardnim CPython konfiguracijama. GIL ograničava istovremeno izvršavanje Python bajtkoda u više niti. Zbog toga povećanje broja niti ne mora dovesti do proporcionalnog ubrzanja kod CPU-bound zadataka. Kod takvih zadataka niti mogu čak biti sporije od sekvencijalnog izvršavanja zbog dodatnog troška upravljanja konkurentnim izvršavanjem.

U praktičnim primjerima ovog rada niti se koriste prvenstveno kod zadataka koji uključuju čekanje. Posebno je značajan primjer simulacije senzorskih očitavanja, gdje svaki senzor zahtijeva određeno vrijeme za očitavanje. U takvoj situaciji niti mogu izvršavati više očitavanja konkurentno, čime se ukupno vrijeme čekanja značajno smanjuje.

2.5.2. Modul multiprocessing

„Modul multiprocessing omogućava izvršavanje programa pomoću više procesa. Za razliku od niti, procesi imaju odvojene memorijske prostore i zasebne interpretere. Zbog toga procesi ne dijele obične Python objekte direktno, već je za razmjenu podataka potrebno koristiti mehanizme međuprocenjske komunikacije. Jedna od najvažnijih prednosti procesa jeste mogućnost korišćenja više procesorskih jezgara za istovremeno izvršavanje računskih zadataka. Ovo je posebno značajno kod CPU-bound operacija, gdje se veliki dio vremena troši na izvršavanje računskih instrukcija“ [2]. Dok kod standardnog rada sa nitima GIL može ograničiti izvršavanje Python bajtkoda, odvojeni procesi mogu izvršavati svoje zadatke nezavisno.

Procesi su zato pogodni za zadatke kao što su složeni matematički proračuni, numeričke simulacije, obrada velikih količina podataka, provjera velikog broja vrijednosti, obrada slike i drugi problemi kod kojih postoji dovoljno posla koji se može podijeliti između više izvršnih jedinica. Međutim, korištenje procesa donosi i određene troškove. Kreiranje procesa zahtijeva više resursa nego kreiranje niti, a komunikacija između procesa je složenija. Kada se podaci šalju iz jednog procesa u drugi, oni se u odgovarajućim slučajevima moraju serijalizovati, prenijeti i ponovo rekonstruisati. Ako se radi sa velikim listama, matricama ili drugim složenim objektima, ovaj proces može zahtijevati značajno vrijeme i memoriju.

Zbog toga veličina zadatka ima veliki uticaj na opravdanost korišćenja procesa. Ako se jedan mali zadatak izvršava veoma brzo, vrijeme potrebno za njegovo slanje drugom procesu može biti veće od vremena potrebnog da se zadatak izvrši direktno u glavnom procesu. U tom slučaju paralelna verzija može biti sporija od sekvencijalne. Jedan od načina za smanjenje ovog problema jeste

grupisanje više manjih zadataka u veće cjeline. Umjesto da se svakom procesu šalje veliki broj pojedinačnih operacija, moguće je formirati veće blokove podataka i svaki blok dodijeliti jednom radnom procesu. Na taj način se smanjuje broj komunikacija između glavnog procesa i radnih procesa.

2.5.3. Modul `concurrent.futures`

„Modul `concurrent.futures` predstavlja viši nivo interfejsa za konkurentno izvršavanje zadataka. Njegova osnovna prednost je u tome što programeru omogućava jednostavniji rad sa nitima i procesima bez potrebe za detaljnim ručnim upravljanjem njihovim životnim ciklusom. Modul pruža dvije osnovne klase: `ThreadPoolExecutor` i `ProcessPoolExecutor`. `ThreadPoolExecutor` koristi bazen niti, dok `ProcessPoolExecutor` koristi bazen procesa. Oba mehanizma omogućavaju da se funkcije i njihovi argumenti jednostavno rasporede radnim jedinicama pomoću metoda kao što su `submit()` i `map()`“ [1]. Kod korišćenja bazena izvršnih jedinica, programer ne mora za svaki pojedinačni zadatak ručno kreirati novu nit ili proces. Umjesto toga, formira se određeni broj radnika koji preuzimaju zadatke iz bazena. Kada jedna radna jedinica završi svoj zadatak, može preuzeti naredni. Ovakav pristup pojednostavljuje organizaciju programa i omogućava bolje upravljanje većim brojem zadataka.

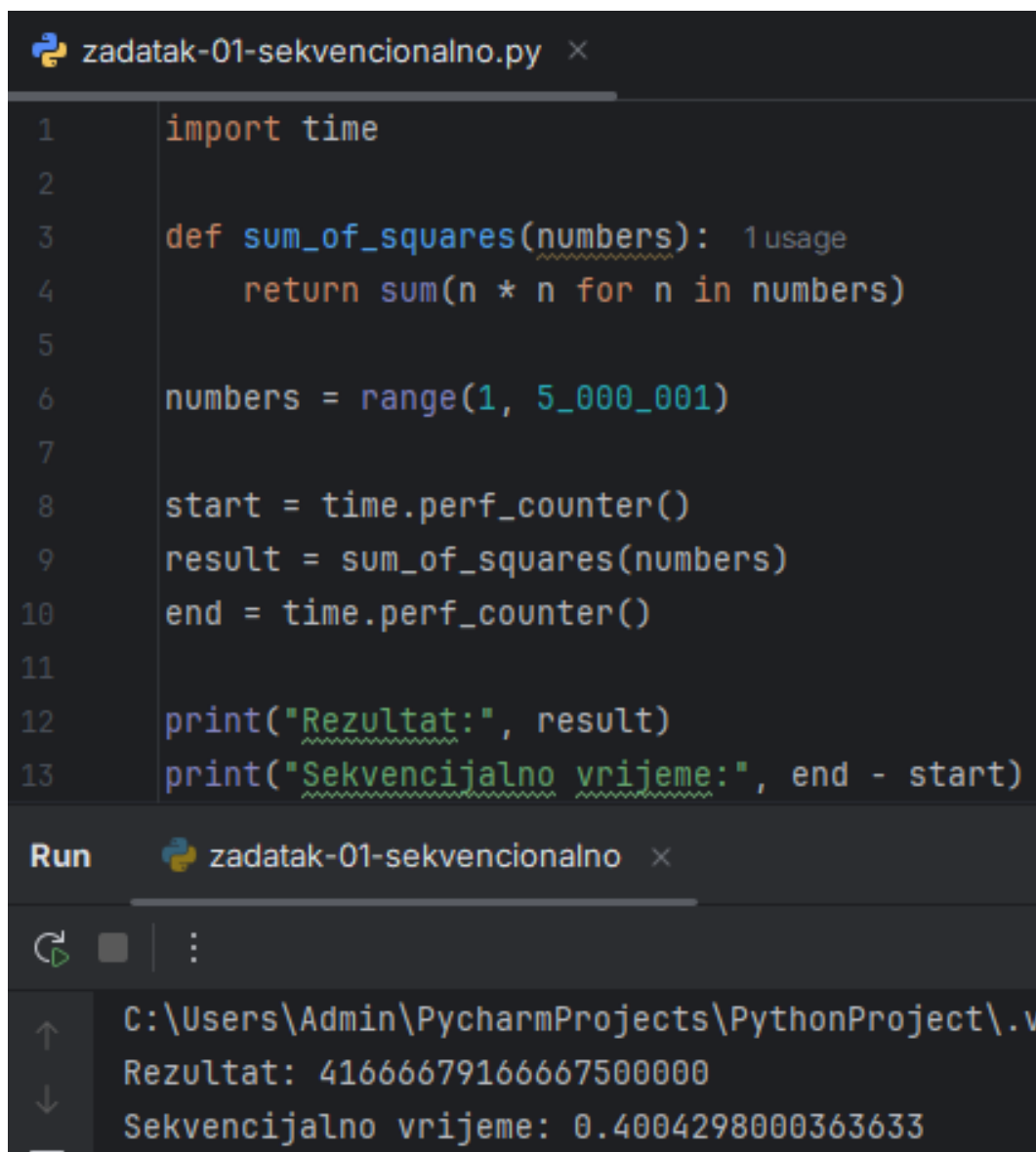
Metoda `submit()` omogućava slanje pojedinačnog zadatka na izvršavanje i vraća objekat `Future`, koji predstavlja rezultat operacije koja će biti dostupna kada zadatak završi. Ovo omogućava programu da nastavi sa drugim poslovima dok se rezultat obrađuje. Sa druge strane, metoda `map()` omogućava jednostavno primjenjivanje iste funkcije nad većim brojem elemenata, pri čemu se izvršavanje može rasporediti između dostupnih radnika. `ThreadPoolExecutor` je naročito koristan kada program ima veći broj nezavisnih I/O-bound zadataka. Primjer može biti slanje više mrežnih zahtjeva ili simulacija očitavanja više senzora. Dok jedna nit čeka na završetak operacije, druga može obrađivati naredni zadatak.

`ProcessPoolExecutor` je namijenjen situacijama u kojima je potrebno koristiti više procesa. Može biti koristan kod CPU-bound zadataka koji se mogu podijeliti na međusobno nezavisne cjeline. Ipak, isti problemi sa prenosom i serijalizacijom podataka koji postoje kod procesnog modela i dalje mogu uticati na performanse. Zbog jednostavnog interfejsa, `concurrent.futures` je posebno pogodan za praktične eksperimente i poređenje sekvencijalnog i paralelnog izvršavanja. U ovom radu koristi se u više primjera upravo zato što omogućava relativno kratku implementaciju paralelne verzije programa, uz zadržavanje jasne strukture koda. Pored jednostavnosti, ovaj modul omogućava lakše eksperimentisanje sa različitim brojem radnika. Parametar `max_workers` određuje koliko niti ili procesa može istovremeno biti aktivno u izvršnom bazenu. Promjenom ove vrijednosti moguće je ispitati kako broj radnih jedinica utiče na ukupno vrijeme izvršavanja.

3. PRAKTIČNA KOMPARACIJA SEKVENCIJALNOG I PARALELNOG KODA

3.1. Primjer 1: Suma kvadrata brojeva

Potrebno je izračunati zbir kvadrata svih brojeva u opsegu od 1 do 5.000.000, odnosno $\sum_{i=1}^{5'000'000} i^2$. Sekvencijalna verzija izračunava sumu direktno nad objektom `range(1, 5_000_001)` unutar glavnog procesa. Paralelna verzija prvo kreira listu svih brojeva u memoriji pomoću `list(range(1, 5_000_001))`, a zatim je dijeli na podnizove (chunks) prema broju dostupnih CPU jezgara. Tako formirani podnizovi proslijeđuju se radnim procesima pomoću `ProcessPoolExecutor().map(sum_of_squares, chunks)`, čime se omogućava paralelna obrada i naknadno sabiranje parcijalnih rezultata u konačnu sumu.



```
zadatak-01-sekvencionalno.py x
1 import time
2
3 def sum_of_squares(numbers): 1 usage
4     return sum(n * n for n in numbers)
5
6 numbers = range(1, 5_000_001)
7
8 start = time.perf_counter()
9 result = sum_of_squares(numbers)
10 end = time.perf_counter()
11
12 print("Rezultat:", result)
13 print("Sekvencijalno vrijeme:", end - start)

Run zadatak-01-sekvencionalno x

C:\Users\Admin\PycharmProjects\PythonProject\.v
Rezultat: 41666679166667500000
Sekvencijalno vrijeme: 0.4004298000363633
```

Slika 1 - Suma kvadrata brojeva - sekvencijalno

```

1  import time
2  from concurrent.futures import ProcessPoolExecutor
3  from multiprocessing import cpu_count
4
5  def sum_of_squares(numbers): 1 usage
6      return sum(n * n for n in numbers)
7
8  if __name__ == "__main__":
9      numbers = list(range(1, 5_000_001))
10     cores = cpu_count()
11     chunk_size = len(numbers) // cores
12     chunks = [numbers[i:i + chunk_size] for i in range(0, len(numbers), chunk_size)]
13
14     start = time.perf_counter()
15     with ProcessPoolExecutor() as executor:
16         partial_results = list(executor.map(sum_of_squares, *iterables: chunks))
17     result = sum(partial_results)
18     end = time.perf_counter()
19
20     print("Rezultat:", result)
21     print("Paralelno vrijeme:", end - start)

```

Run zadatak-01-paralelno

```

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\Py
Rezultat: 41666679166667500000
Paralelno vrijeme: 0.6941026999847963

```

Slika 2 - Suma kvadrata brojeva - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.4004298000363633}{0.6941026999847963} \approx \mathbf{0.576903}$$

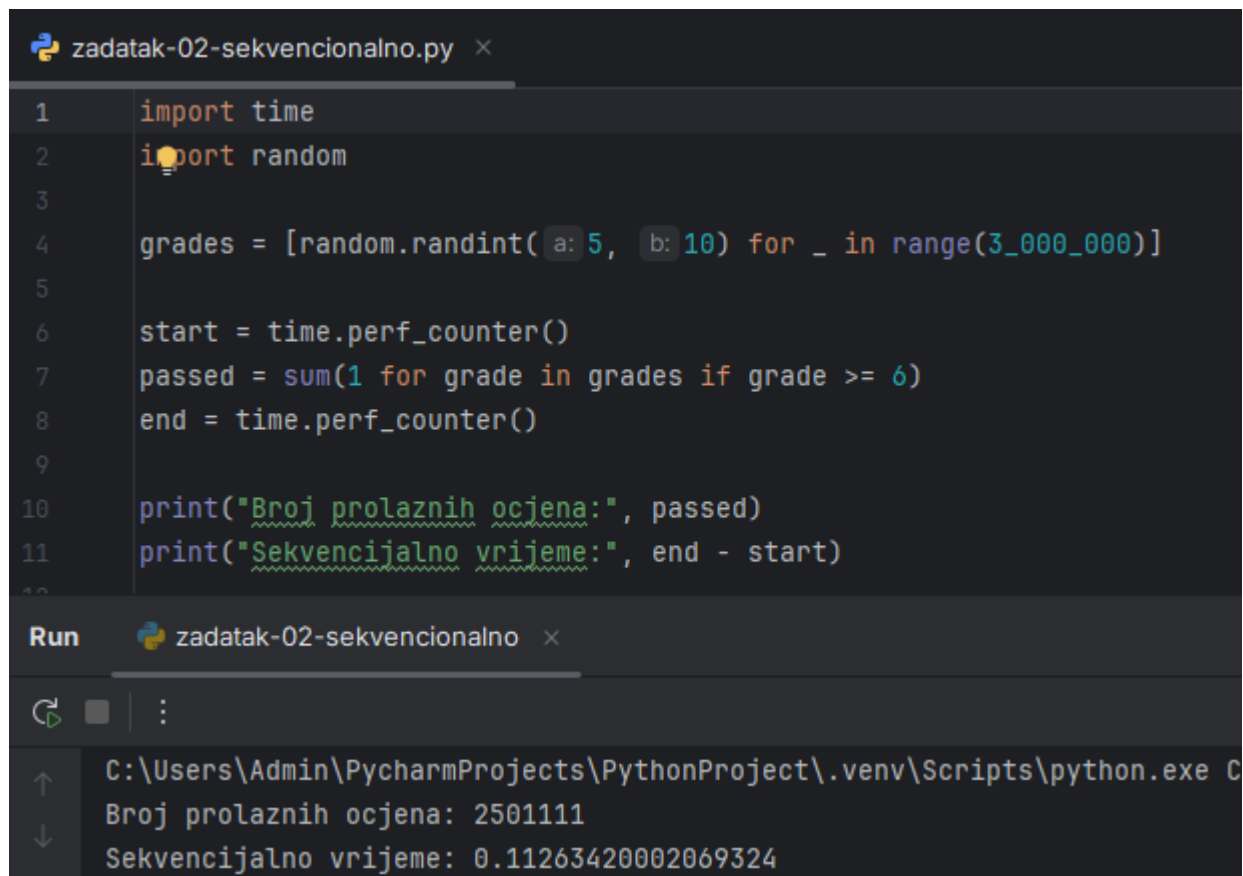
Opseg brojeva (N)	Izračunat rezultat	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
5.000.000	41666679166667500000	0.400	0.694	0.577

Na testiranom opsegu od 5 miliona elemenata, sekvencijalna verzija ostvarila je bolje vrijeme izvršavanja od paralelne verzije. Sekvencijalno izračunavanje trajalo je 0,400 sekundi, dok je paralelnoj verziji bilo potrebno 0,694 sekundi. Na osnovu ovih rezultata, ubrzanje iznosi 0,577, što znači da je paralelna verzija bila približno 42 % sporija od sekvencijalne.

Paralelizacija izračunavanja sume kvadrata pomoću eksplicitnih podnizova (chunks) ne donosi ubrzanje na opsegu od 5 miliona elemenata zbog velikog overhead-a međuprocene komunikacije (IPC). Kako bi se postiglo stvarno ubrzanje ($S > 1$), paralelnoj funkciji trebalo bi prosljeđivati samo početni i krajnji indeks opsega, odnosno par (start, end), umjesto cijele liste. Na taj način izbjeglo bi se nepotrebno kopiranje miliona elemenata u memoriji i smanjilo opterećenje nastalo prenosom podataka između procesa.

3.2. Primjer 2: Obrada liste ocjena

Potrebno je izračunati koliko ocjena iz generisane liste od 3.000.000 nasumičnih elemenata, pri čemu se ocjene nalaze u opsegu od 5 do 10, zadovoljava uslov prolaznosti (≥ 6). Sekvencijalna verzija prebrojava prolazne ocjene u glavnom procesu pomoću generatorskog izraza `sum(1 for grade in grades if grade >= 6)`. Paralelna verzija dijeli listu ocjena na jednake dijelove (chunks) prema broju dostupnih CPU jezgara, zatim ih prosljeđuje radnim procesima pomoću `ProcessPoolExecutor().map(count_passed, chunks)`. Nakon završetka obrade, parcijalni rezultati svih procesa se sabiraju kako bi se dobio ukupan broj prolaznih ocjena.



```
zadatak-02-sekvencionalno.py x
1 import time
2 import random
3
4 grades = [random.randint(a=5, b=10) for _ in range(3_000_000)]
5
6 start = time.perf_counter()
7 passed = sum(1 for grade in grades if grade >= 6)
8 end = time.perf_counter()
9
10 print("Broj prolaznih ocjena:", passed)
11 print("Sekvencijalno vrijeme:", end - start)
12
Run zadatak-02-sekvencionalno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C
Broj prolaznih ocjena: 2501111
Sekvencijalno vrijeme: 0.11263420002069324
```

Slika 3 - Obrada liste ocjena - sekvencionalno

```

1  > import ...
5
6  def count_passed(chunk): 1usage
7      return sum(1 for grade in chunk if grade >= 6)
8
9  if __name__ == "__main__":
10     grades = [random.randint(a: 5, b: 10) for _ in range(3_000_000)]
11     cores = cpu_count()
12     chunk_size = len(grades) // cores
13     chunks = [grades[i:i + chunk_size] for i in range(0, len(grades), chunk_size)]
14
15     start = time.perf_counter()
16     with ProcessPoolExecutor() as executor:
17         results = list(executor.map(count_passed, *iterables: chunks))
18     passed = sum(results)
19     end = time.perf_counter()
20
21     print("Broj prolaznih ocjena:", passed)
22     print("Paralelno vrijeme:", end - start)
23
Run  zadatak-02-paralelno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\
Broj prolaznih ocjena: 2500018
Paralelno vrijeme: 0.3885924000060186

```

Slika 4 - Obrada liste ocjena - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.11263420002069324}{0.3885924000060186} \approx \mathbf{0.289852}$$

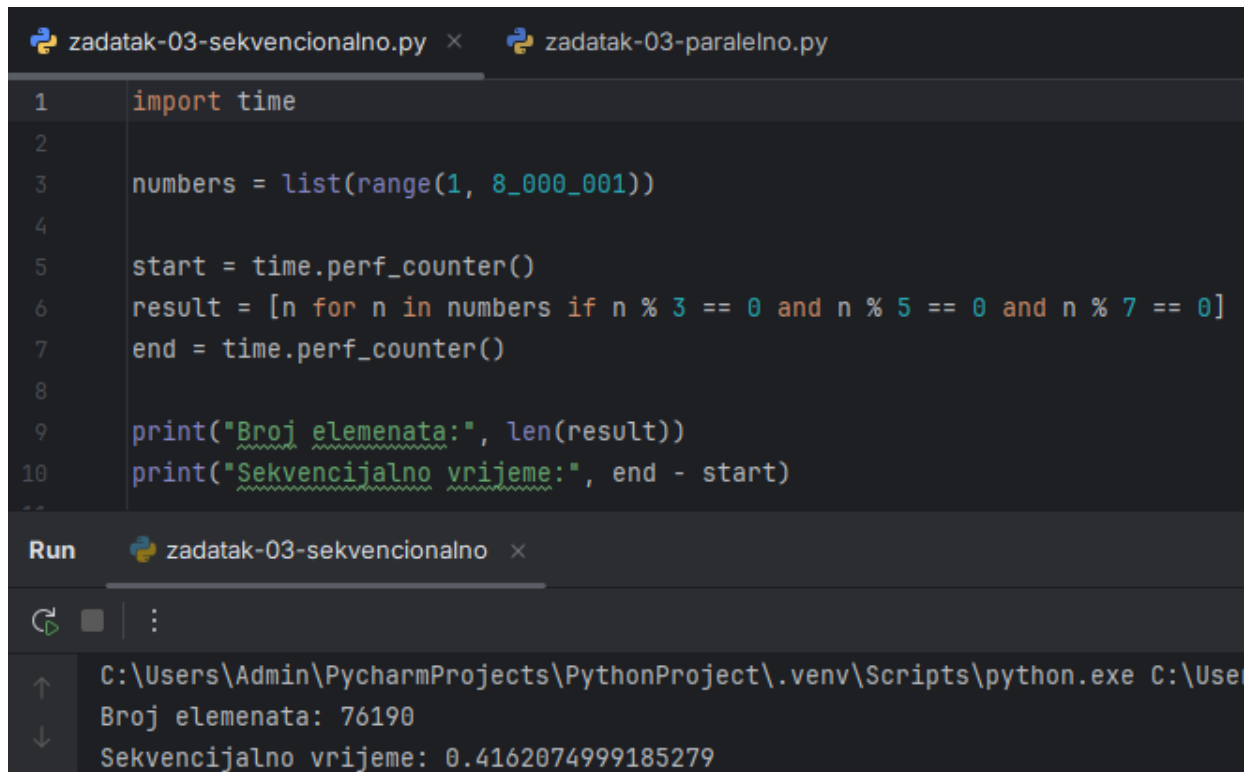
Ukupan broj ocjena	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
3.000.000	0.113	0.389	0.290

Na testiranom skupu od 3 miliona ocjena, sekvencijalna verzija ostvarila je više nego trostruko bolje vrijeme izvršavanja u odnosu na paralelnu verziju. Sekvencijalno prebrojavanje trajalo je svega 0,113 sekundi, dok je paralelnoj verziji bilo potrebno 0,389 sekundi. Na osnovu ovih rezultata, ubrzanje iznosi 0,290, što pokazuje značajno usporavanje paralelne verzije.

Prebrojavanje elemenata pomoću jednostavnog uslova predstavlja tipičan primjer operacije ograničene brzinom pristupa memoriji (memory-bound task). Korištenje multiprocessing pristupa, odnosno ProcessPoolExecutor-a, u ovom slučaju uvodi dodatni overhead zbog komunikacije između procesa, čime se gubi korist od paralelizacije. Zbog toga sekvencijalni pristup pokazuje veću efikasnost. Sličan efekat može se postići i korištenjem vektorizovanih biblioteka, poput NumPy-ja, koje omogućavaju efikasnu obradu podataka smještenih u memorijskom bloku.

3.3. Primjer 3: Provjera djeljivosti velikog broja elemenata

Potrebno je filtrirati niz od 8.000.000 cijelih brojeva, u opsegu od 1 do 8.000.000, i izdvojiti samo one brojeve koji su istovremeno djeljivi sa 3, 5 i 7, odnosno sa 105. Sekvencijalna verzija prolazi kroz cijelu listu pomoću list comprehension strukture unutar glavnog procesa. Paralelna verzija prvo kreira kompletnu listu pomoću `list(range(1, 8_000_001))`, zatim je dijeli na podnizove (chunks) prema broju dostupnih CPU jezgara. Podnizovi se proslijeđuju radnim procesima pomoću `ProcessPoolExecutor().map(filter_numbers, chunks)`, nakon čega se filtrirani rezultati svih procesa spajaju u jednu listu pomoću metode `extend()`.



```
zadatak-03-sekvencionalno.py x zadatak-03-paralelno.py
1 import time
2
3 numbers = list(range(1, 8_000_001))
4
5 start = time.perf_counter()
6 result = [n for n in numbers if n % 3 == 0 and n % 5 == 0 and n % 7 == 0]
7 end = time.perf_counter()
8
9 print("Broj elemenata:", len(result))
10 print("Sekvencijalno vrijeme:", end - start)
11
Run zadatak-03-sekvencionalno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Use
Broj elemenata: 76190
Sekvencijalno vrijeme: 0.4162074999185279
```

Slika 5 - Provjera djeljivosti velikog broja elemenata - sekvencionalno

```

1 import time
2 from concurrent.futures import ProcessPoolExecutor
3 from multiprocessing import cpu_count
4
5 def filter_numbers(chunk):
6     return [n for n in chunk if n % 3 == 0 and n % 5 == 0 and n % 7 == 0]
7
8 if __name__ == "__main__":
9     numbers = list(range(1, 8_000_001))
10    cores = cpu_count()
11    chunk_size = len(numbers) // cores
12    chunks = [numbers[i:i + chunk_size] for i in range(0, len(numbers), chunk_size)]
13
14    start = time.perf_counter()
15    with ProcessPoolExecutor() as executor:
16        parts = list(executor.map(filter_numbers, chunks))
17    result = []
18    for part in parts:
19        result.extend(part)
20    end = time.perf_counter()
21
22    print("Broj elemenata:", len(result))
23    print("Paralelno vrijeme:", end - start)

```

Run zadatak-03-paralelno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\Py
 Broj elemenata: 76190
 Paralelno vrijeme: 0.9416315000271425

Slika 6 - Provjera djeljivosti velikog broja elemenata - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.4162074999185279}{0.9416315000271425} \approx \mathbf{0.441901}$$

Ukupan broj elemenata	Pronađeno elemenata	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
8.000.000	76.190	0.416	0.942	0.442

Na testiranom skupu od 8 miliona elemenata, sekvencijalna verzija ostvarila je više nego dvostruko bolje vrijeme izvršavanja u odnosu na paralelnu verziju. Sekvencijalno filtriranje trajalo je 0,416 sekundi, dok je paralelnoj verziji bilo potrebno 0,942 sekunde. Na osnovu ovih rezultata, ubrzanje iznosi 0,442, što ukazuje na značajno usporavanje paralelne verzije.

Filtriranje brojeva pomoću jednostavnih matematičkih ili logičkih uslova predstavlja tipičan primjer operacije ograničene brzinom pristupa memoriji (memory-bound task). Korištenje ProcessPoolExecutor-a uz prenošenje čitavih listi između procesa uvodi veliki interprocesni komunikacioni overhead, zbog čega se gubi potencijalna korist od paralelizacije. Zbog toga je sekvencijalni pristup u ovom slučaju efikasnije rješenje. Dodatno poboljšanje može se postići korištenjem optimizovanih biblioteka poput NumPy-ja, koje omogućavaju efikasnu obradu podataka u jedinstvenom memorijskom bloku bez nepotrebnog kopiranja.

3.4. Primjer 4: Simulacija više sporih operacija

Potrebno je izvršiti 5 sporih i međusobno nezavisnih operacija, pri čemu svaka traje tačno 2 sekunde. U ovom primjeru koristi se `time.sleep(2)` kako bi se simuliralo čekanje na eksterne resurse, kao što su I/O operacije, mrežni HTTP zahtjevi ili upiti prema bazi podataka. Sekvencijalna verzija izvršava operacije jednu za drugom unutar jedne niti glavnog procesa. Paralelna verzija koristi `ThreadPoolExecutor(max_workers=5)` iz modula `concurrent.futures`, čime se operacije raspoređuju na 5 zasebnih niti koje se izvršavaju istovremeno.

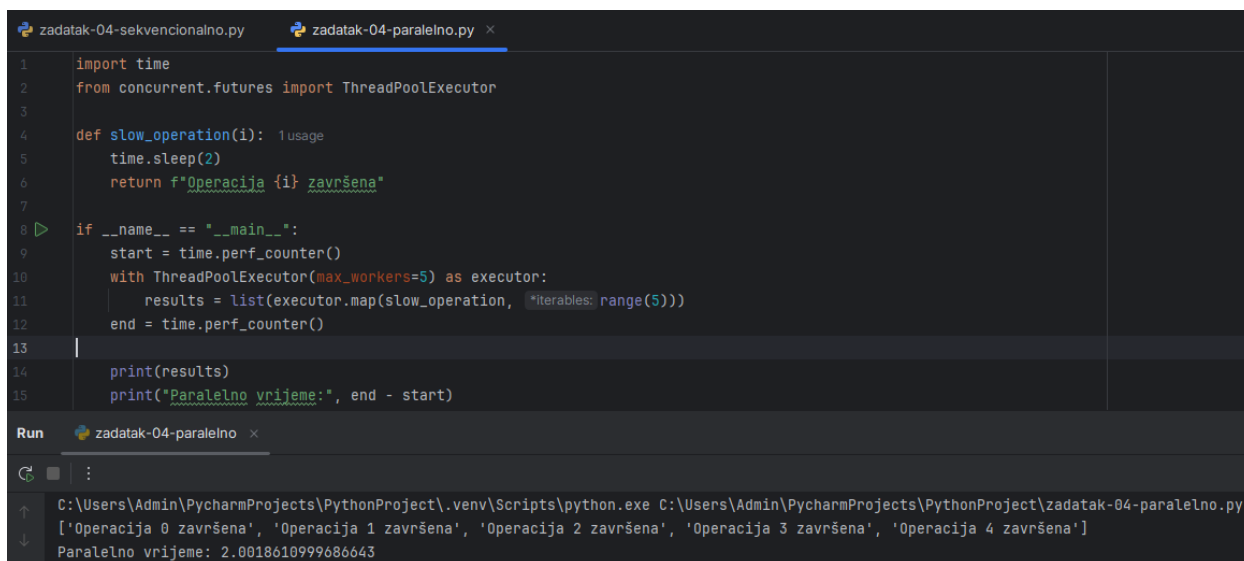


```
1 import time
2
3 def slow_operation(i):
4     time.sleep(2)
5     return f"Operacija {i} završena"
6
7 start = time.perf_counter()
8 results = [slow_operation(i) for i in range(5)]
9 end = time.perf_counter()
10
11 print(results)
12 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-04-sekvencionalno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-04-sekvencionalno.py
['Operacija 0 završena', 'Operacija 1 završena', 'Operacija 2 završena', 'Operacija 3 završena', 'Operacija 4 završena']
Sekvencijalno vrijeme: 10.00222499994561

Slika 7 - Simulacija više sporih operacija - sekvencionalno



```
1 import time
2 from concurrent.futures import ThreadPoolExecutor
3
4 def slow_operation(i):
5     time.sleep(2)
6     return f"Operacija {i} završena"
7
8 if __name__ == "__main__":
9     start = time.perf_counter()
10    with ThreadPoolExecutor(max_workers=5) as executor:
11        results = list(executor.map(slow_operation, *iterables: range(5)))
12    end = time.perf_counter()
13
14    print(results)
15    print("Paralelno vrijeme:", end - start)
```

Run zadatak-04-paralelno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-04-paralelno.py
['Operacija 0 završena', 'Operacija 1 završena', 'Operacija 2 završena', 'Operacija 3 završena', 'Operacija 4 završena']
Paralelno vrijeme: 2.0018610999686643

Slika 8 - Simulacija više sporih operacija - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{10.00222499994561}{2.0018610999686643} \approx 4.996463$$

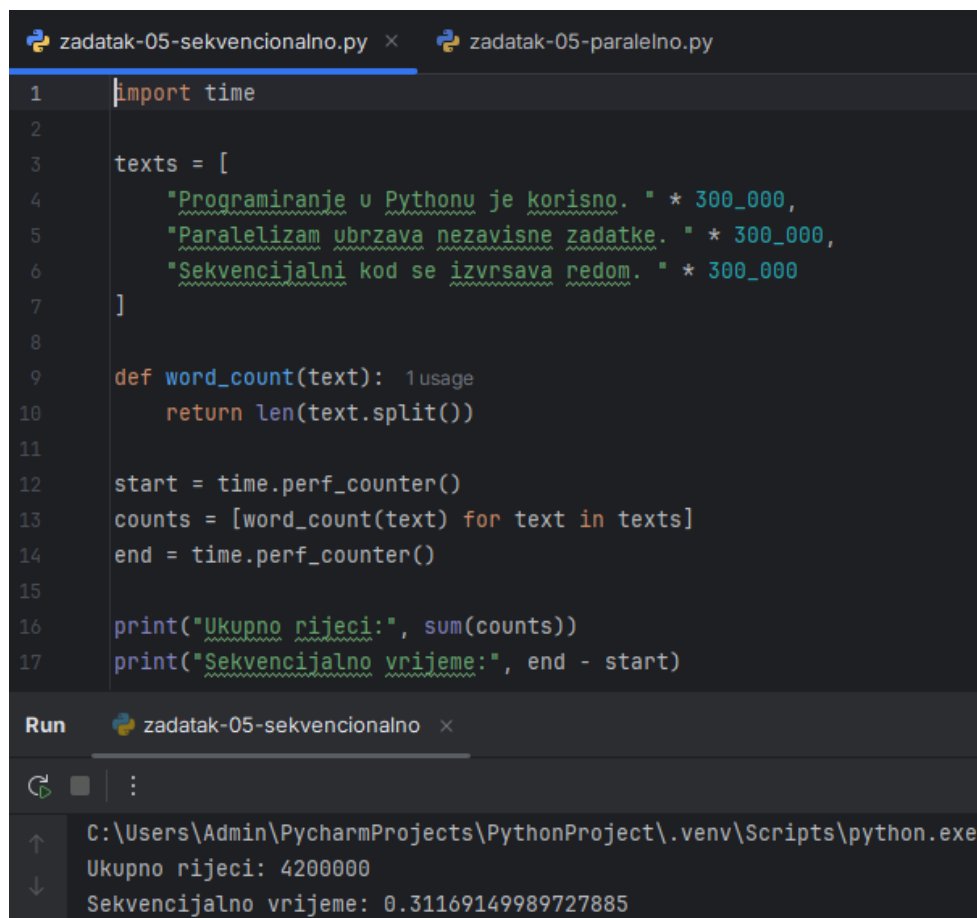
Broj operacija	Trajanje pojedinačne operacije	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
5	2.0 s	10.002	2.002	4.996

Na testiranom primjeru, paralelna verzija sa nitima ostvarila je gotovo teorijski maksimalno ubrzanje od 5 puta u odnosu na sekvencijalnu verziju. Sekvencijalno izvršavanje 5 operacija trajalo je 10,002 sekunde, dok je konkurentno izvršavanje pomoću niti trajalo svega 2,002 sekunde. Na osnovu toga, ostvareno ubrzanje iznosi 4,996.

Konkurentno izvršavanje pomoću ThreadPoolExecutor-a predstavlja veoma pogodan pristup za operacije ograničene ulazno-izlaznim operacijama (I/O-bound tasks). Za razliku od računski intenzivnih zadataka (CPU-bound), kod kojih se za efikasnu paralelizaciju najčešće koriste odvojeni procesi, I/O operacije uglavnom provode vrijeme čekajući odgovor eksternih resursa. Zbog toga višenitni pristup u ovakvim slučajevima može ostvariti gotovo linearno ubrzanje, proporcionalno broju angažovanih niti.

3.5. Primjer 5: Analiza više tekstualnih dokumenata

Potrebno je prebrojati ukupan broj riječi u 3 velika tekstualna dokumenta. Svaki tekst sastoji se od iste rečenice ponovljene 300.000 puta, što ukupno daje 4.200.000 riječi. Sekvencijalna verzija obrađuje tekstove redom, unutar jedne niti glavnog procesa, pri čemu se za brojanje riječi koristi funkcija `len(text.split())`. Paralelna verzija raspoređuje 3 velika teksta na zasebne radne procese pomoću `ProcessPoolExecutor().map(word_count, texts)`. Nakon završetka obrade, rezultati dobijeni za pojedinačne tekstove sabiraju se kako bi se dobio ukupan broj riječi.



```
zadatak-05-sekvencionalno.py x zadatak-05-paralelno.py
1 import time
2
3 texts = [
4     "Programiranje u Pythonu je korisno. " * 300_000,
5     "Paralelizam ubrzava nezavisne zadatke. " * 300_000,
6     "Sekvencijalni kod se izvrsava redom. " * 300_000
7 ]
8
9 def word_count(text): 1 usage
10     return len(text.split())
11
12 start = time.perf_counter()
13 counts = [word_count(text) for text in texts]
14 end = time.perf_counter()
15
16 print("Ukupno rijeci:", sum(counts))
17 print("Sekvencijalno vrijeme:", end - start)

Run zadatak-05-sekvencionalno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Ukupno rijeci: 4200000
Sekvencijalno vrijeme: 0.31169149989727885
```

Slika 9 - Analiza više tekstualnih dokumenata - sekvencionalno

```

1  import time
2  from concurrent.futures import ProcessPoolExecutor
3
4  def word_count(text):
5      return len(text.split())
6
7  if __name__ == "__main__":
8      texts = [
9          "Programiranje u Pythonu je korisno. " * 300_000,
10         "Paralelizam ubrzava nezavisne zadatke. " * 300_000,
11         "Sekvencijalni kod se izvršava redom. " * 300_000
12     ]
13
14     start = time.perf_counter()
15     with ProcessPoolExecutor() as executor:
16         counts = list(executor.map(word_count, *iterables: texts))
17     end = time.perf_counter()
18
19     print("Ukupno riječi:", sum(counts))
20     print("Paralelno vrijeme:", end - start)

```

Run zadatak-05-paralelno

```

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.
Ukupno riječi: 4200000
Paralelno vrijeme: 0.3954547999892384

```

Slika 10 - Analiza više tekstualnih dokumenata - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.31169149989727885}{0.3954547999892384} \approx \mathbf{0.788185}$$

Broj dokumenata	Ukupno prebrojano riječi	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
3	4.200.000	0.312	0.395	0.788

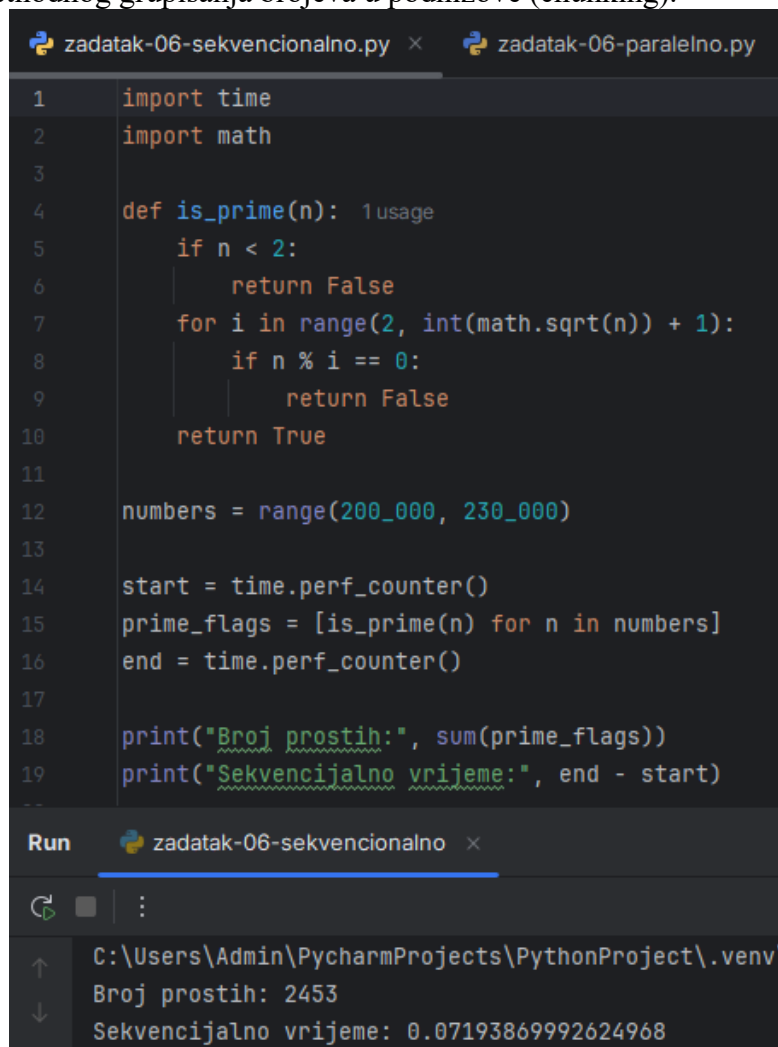
Na testiranom skupu od 3 velika tekstualna dokumenta, sekvencijalna verzija ostvarila je bolje vrijeme izvršavanja u odnosu na paralelnu verziju. Sekvencijalno prebrojavanje trajalo je 0,312

sekundi, dok je paralelnoj verziji bilo potrebno 0,395 sekundi. Na osnovu ovih rezultata, ostvareno ubrzanje iznosi 0,788, što predstavlja usporavanje od oko 21%.

Paralelizacija obrade teksta slanjem cijelih stringova radnim procesima nije efikasna zbog troška serijalizacije i prenosa velike količine podataka između procesa. Kako bi se postiglo stvarno ubrzanje ($S > 1$), praktičnije rješenje je radnim procesima proslijediti samo putanje do datoteka na disku. Na taj način svaki proces može samostalno otvoriti i pročitati svoj fajl, čime se izbjegava nepotrebno prenošenje velikih tekstualnih blokova kroz međuprocenu komunikaciju.

3.6. Primjer 6: Računanje prostih brojeva

Potrebno je odrediti koliko brojeva u opsegu od 200.000 do 230.000, odnosno ukupno 30.000 elemenata, ima osobinu prostog broja. Za svaki broj provjerava se djeljivost sa svim brojevima do njegovog kvadratnog korijena, pri čemu se koristi funkcija `math.sqrt(n)`. Sekvencijalna verzija prolazi kroz cijeli raspon brojeva redom, unutar jedne niti glavnog procesa. Paralelna verzija prosljeđuje brojeve radnim procesima pojedinačno pomoću `ProcessPoolExecutor().map(is_prime, numbers)`, bez prethodnog grupisanja brojeva u podnizove (chunking).



```
zadatak-06-sekvencionalno.py x zadatak-06-paralelno.py
1 import time
2 import math
3
4 def is_prime(n): 1 usage
5     if n < 2:
6         return False
7     for i in range(2, int(math.sqrt(n)) + 1):
8         if n % i == 0:
9             return False
10    return True
11
12 numbers = range(200_000, 230_000)
13
14 start = time.perf_counter()
15 prime_flags = [is_prime(n) for n in numbers]
16 end = time.perf_counter()
17
18 print("Broj prostih:", sum(prime_flags))
19 print("Sekvencijalno vrijeme:", end - start)
--
Run zadatak-06-sekvencionalno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\
Broj prostih: 2453
Sekvencijalno vrijeme: 0.07193869992624968
```

Slika 11 - Računanje prostih brojeva - sekvencijalno

```

1 import time
2 import math
3 from concurrent.futures import ProcessPoolExecutor
4
5 def is_prime(n):
6     if n < 2:
7         return False
8     for i in range(2, int(math.sqrt(n)) + 1):
9         if n % i == 0:
10            return False
11    return True
12
13 if __name__ == "__main__":
14     numbers = list(range(200_000, 230_000))
15
16     start = time.perf_counter()
17     with ProcessPoolExecutor() as executor:
18         prime_flags = list(executor.map(is_prime, numbers))
19     end = time.perf_counter()
20
21     print("Broj prostih:", sum(prime_flags))
22     print("Paralelno vrijeme:", end - start)

```

Run zadatak-06-paralelno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C
 Broj prostih: 2453
 Paralelno vrijeme: 7.999890799983405

Slika 12 - Računanje prostih brojeva - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.07193869992624968}{7.999890799983405} \approx \mathbf{0.008992}$$

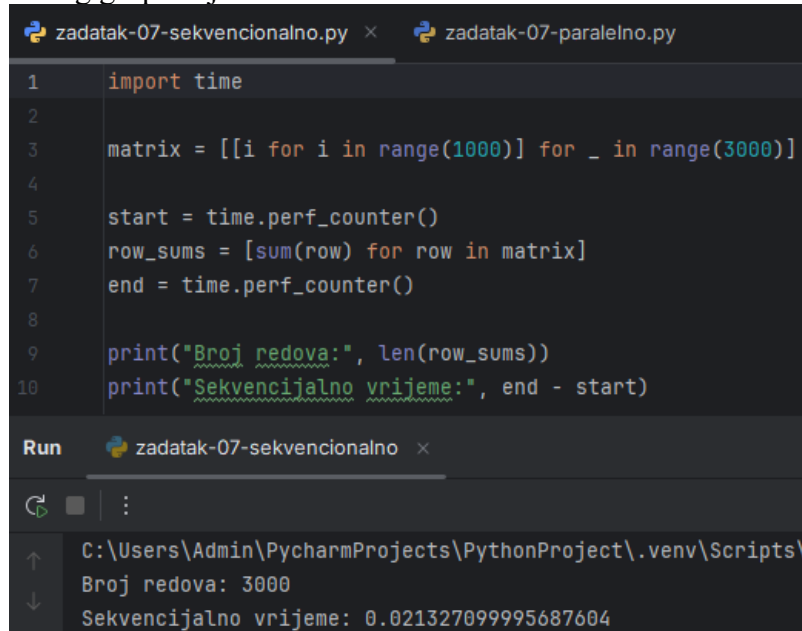
Interval brojeva	Ukupno prostih brojeva	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
200.000 – 230.000	2.453	0.072	8.000	0.009

Na testiranom opsegu od 30.000 elemenata, sekvencijalna verzija ostvarila je značajno bolje vrijeme izvršavanja u odnosu na paralelnu verziju. Sekvencijalna provjera trajala je svega 0,072 sekunde, dok je paralelnoj verziji bilo potrebno čak 8,000 sekundi. Na osnovu ovih rezultata, ostvareno ubrzanje iznosi 0,009, što znači da je paralelna verzija bila više od 110 puta sporija.

Provjera prostih brojeva predstavlja CPU-bound zadatak koji je pogodan za paralelnu obradu, ali samo ako se posao pravilno raspodijeli između procesa. U ovom slučaju, svaki broj je pojedinačno prosljeđen radnim procesima, što je stvorilo veliki interprocesni overhead i značajno usporilo izvršavanje. Efikasnije rješenje bilo bi grupisanje brojeva u veće podnizove (chunks) ili korištenje odgovarajućeg chunksize parametra unutar .map() poziva. Na taj način bi se smanjio broj komunikacija između procesa i omogućilo da procesori veći dio vremena provode na samom računanju umjesto na prenosu podataka.

3.7. Primjer 7: Paralelna obrada redova matrice

Potrebno je izračunati sumu elemenata za svaki red matrice dimenzija 3.000×1.000 , što ukupno predstavlja 3.000.000 elemenata. Sekvencijalna verzija obrađuje redove jedan za drugim unutar glavnog procesa, koristeći izraz `[sum(row) for row in matrix]`. Paralelna verzija raspoređuje pojedinačne redove matrice na radne procese pomoću `ProcessPoolExecutor().map(sum_row, matrix)`, bez prethodnog grupisanja redova u veće blokove.



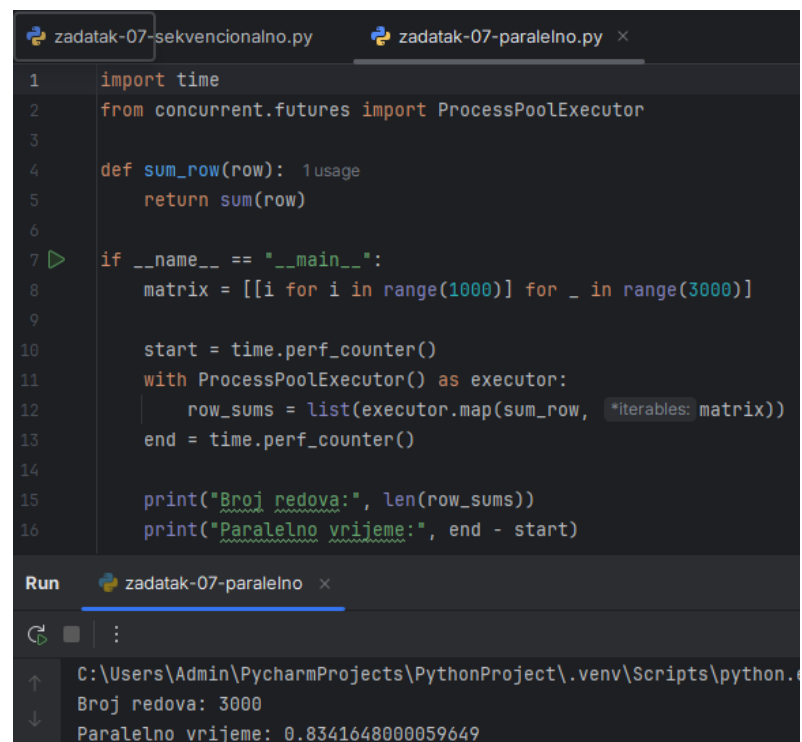
The screenshot shows a PyCharm IDE window with two tabs: 'zadatak-07-sekvencionalno.py' and 'zadatak-07-paralelno.py'. The 'zadatak-07-sekvencionalno.py' tab is active, displaying the following Python code:

```
1 import time
2
3 matrix = [[i for i in range(1000)] for _ in range(3000)]
4
5 start = time.perf_counter()
6 row_sums = [sum(row) for row in matrix]
7 end = time.perf_counter()
8
9 print("Broj redova:", len(row_sums))
10 print("Sekvencijalno vrijeme:", end - start)
```

Below the code editor, the 'Run' button is visible, and the output console shows the following results:

```
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj redova: 3000
Sekvencijalno vrijeme: 0.021327099995687604
```

Slika 13 - Paralelna obrada redova matrice - sekvencionalno



The screenshot shows the same PyCharm IDE window, but now the 'zadatak-07-paralelno.py' tab is active. The code is as follows:

```
1 import time
2 from concurrent.futures import ProcessPoolExecutor
3
4 def sum_row(row):
5     return sum(row)
6
7 if __name__ == "__main__":
8     matrix = [[i for i in range(1000)] for _ in range(3000)]
9
10    start = time.perf_counter()
11    with ProcessPoolExecutor() as executor:
12        row_sums = list(executor.map(sum_row, matrix))
13    end = time.perf_counter()
14
15    print("Broj redova:", len(row_sums))
16    print("Paralelno vrijeme:", end - start)
```

The output console shows the following results:

```
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj redova: 3000
Paralelno vrijeme: 0.8341648000059649
```

Slika 14 - Paralelna obrada redova matrice - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.021327099995687604}{0.8341648000059649} \approx \mathbf{0.025567}$$

Dimenzije matrice	Ukupno obrađeno redova	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
3.000 × 1.000	3.000	0.021	0.834	0.026

Na testiranoj matrici dimenzija 3.000 × 1.000, sekvencijalna verzija ostvarila je značajno bolje vrijeme izvršavanja u odnosu na paralelnu verziju. Sekvencijalno sabiranje svih redova trajalo je svega 0,021 sekundu, dok je paralelnoj verziji bilo potrebno 0,834 sekunde. Na osnovu ovih rezultata, ostvareno ubrzanje iznosi 0,026, što znači da je paralelna verzija bila više od 39 puta sporija.

Paralelna obrada redova matrice pomoću ProcessPoolExecutor-a i ugrađenih Python lista u ovom slučaju uvodi veliki komunikacioni overhead. Svaki red se mora serijalizovati, prenijeti drugom procesu i zatim ponovo obraditi, što značajno povećava ukupno vrijeme izvršavanja. Za efikasnu obradu velikih matrica u praksi se često koristi biblioteka NumPy, koja podatke čuva u kontinualnom memorijskom bloku i omogućava znatno efikasnije operacije nad velikim skupovima numeričkih podataka.

3.8. Primjer 8: Filtriranje podataka iz velike liste

Potrebno je iz liste od 12.000.000 elemenata, koja sadrži brojeve od 1 do 12.000.000, izdvojiti sve elemente koji su veći od 6.000.000 i istovremeno parni, odnosno koji zadovoljavaju uslov $x > 6_000_000$ and $x \% 2 == 0$. Sekvencijalna verzija koristi standardno razumijevanje listi (list comprehension) unutar glavnog procesa. Paralelna verzija dijeli početnu listu na blokove (chunks) prema broju dostupnih procesorskih jezgara dobijenom pomoću `cpu_count()`. Dobijeni blokovi se zatim obrađuju konkurentno pomoću ProcessPoolExecutor()-a, nakon čega se filtrirane podliste spajaju u jednu konačnu listu.

```

1 import time
2
3 data = list(range(1, 12_000_001))
4
5 start = time.perf_counter()
6 filtered = [x for x in data if x > 6_000_000 and x % 2 == 0]
7 end = time.perf_counter()
8
9 print("Broj filtriranih elemenata:", len(filtered))
10 print("Sekvencijalno vrijeme:", end - start)
11

```

Run zadatak-08-sekvencionalno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
 Broj filtriranih elemenata: 3000000
 Sekvencijalno vrijeme: 0.4589343999978155

Slika 15 - Filtriranje podataka iz velike liste - sekvencionalno

```

1 import time
2 from concurrent.futures import ProcessPoolExecutor
3 from multiprocessing import cpu_count
4
5 def filter_chunk(chunk):
6     return [x for x in chunk if x > 6_000_000 and x % 2 == 0]
7
8 if __name__ == "__main__":
9     data = list(range(1, 12_000_001))
10    cores = cpu_count()
11    chunk_size = len(data) // cores
12    chunks = [data[i:i + chunk_size] for i in range(0, len(data), chunk_size)]
13
14    start = time.perf_counter()
15    with ProcessPoolExecutor() as executor:
16        parts = list(executor.map(filter_chunk, chunks))
17    filtered = []
18    for part in parts:
19        filtered.extend(part)
20    end = time.perf_counter()
21
22    print("Broj filtriranih elemenata:", len(filtered))
23    print("Paralelno vrijeme:", end - start)

```

Run zadatak-08-paralelno

```

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Python
Broj filtriranih elemenata: 3000000
Paralelno vrijeme: 1.6345101000042632

```

Slika 16 - Filtriranje podataka iz velike liste - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.4589343999978155}{1.6345101000042632} \approx \mathbf{0.280778}$$

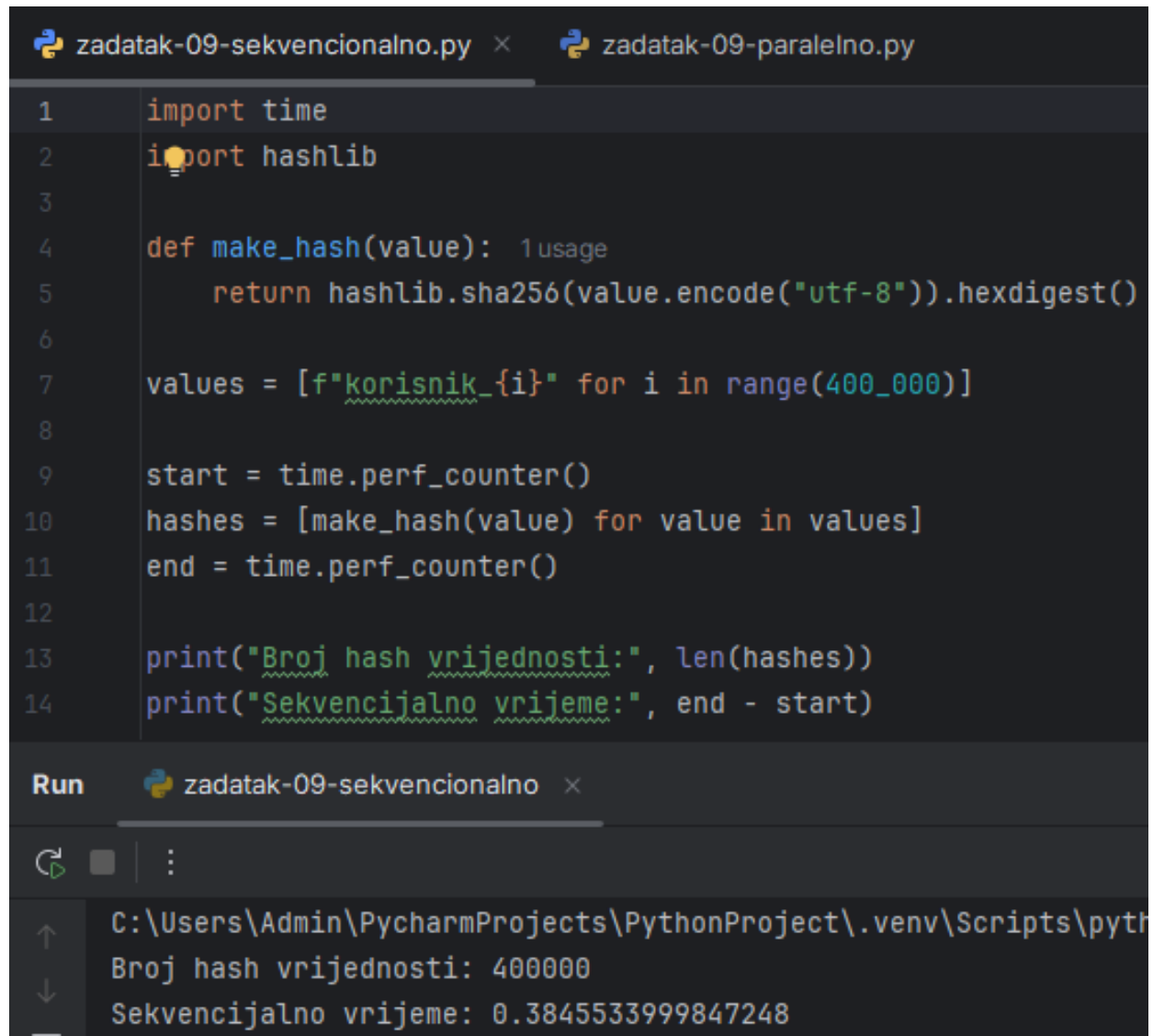
Ukupan broj elemenata	Broj filtriranih elemenata	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
12.000.000	3.000.000	0.459	1.635	0.281

Na skupu od 12.000.000 elemenata, sekvencijalna verzija ostvarila je znatno bolje vrijeme izvršavanja u odnosu na paralelnu verziju. Sekvencijalno filtriranje trajalo je 0,459 sekundi, dok je paralelnoj verziji bilo potrebno 1,635 sekundi. Na osnovu ovih rezultata, ostvareno ubrzanje iznosi 0,281, što znači da je paralelna verzija bila više od 3,5 puta sporija.

Filtriranje jednostavnih nizova u memoriji predstavlja primjer operacije kod koje je računski rad po elementu premali u odnosu na troškove serijalizacije i komunikacije između procesa. Zbog toga paralelizacija u ovom slučaju ne donosi očekivano ubrzanje. Ona ima više smisla kada je obrada svakog elementa računski zahtjevnija, kao što su složene matematičke transformacije ili obrada slika. Za rad sa velikim numeričkim skupovima podataka efikasnije mogu biti vektorizovane biblioteke poput NumPy-ja i Pandas-a, koje smanjuju potrebu za komunikacijom između Python procesa.

3.9. Primjer 9: Generisanje SHA-256 hash vrijednosti

Potrebno je izračunati SHA-256 hash vrijednost za 400.000 kratkih stringova, generisanih u formatu korisnik_*i*. Sekvencijalna verzija izračunava hash vrijednosti redom, unutar glavne niti, koristeći razumijevanje listi (list comprehension). Paralelna verzija koristi ProcessPoolExecutor() sa definisanim parametrom chunksize=1000, čime se elementi obrađuju u grupama. Na ovaj način smanjuje se trošak pojedinačnog prenosa podataka između procesa i omogućava efikasnije raspoređivanje posla među radnim procesima.



```
zadatak-09-sekvencionalno.py x zadatak-09-paralelno.py
1 import time
2 import hashlib
3
4 def make_hash(value):
5     return hashlib.sha256(value.encode("utf-8")).hexdigest()
6
7 values = [f"korisnik_{i}" for i in range(400_000)]
8
9 start = time.perf_counter()
10 hashes = [make_hash(value) for value in values]
11 end = time.perf_counter()
12
13 print("Broj hash vrijednosti:", len(hashes))
14 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-09-sekvencionalno x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj hash vrijednosti: 400000
Sekvencijalno vrijeme: 0.3845533999847248

Slika 17 - Generisanje SHA-256 hash vrijednosti - sekvencionalno

```

1 import time
2 import hashlib
3 from concurrent.futures import ProcessPoolExecutor
4
5 def make_hash(value):
6     return hashlib.sha256(value.encode("utf-8")).hexdigest()
7
8 if __name__ == "__main__":
9     values = [f"korisnik_{i}" for i in range(400_000)]
10
11     start = time.perf_counter()
12
13     with ProcessPoolExecutor() as executor:
14         hashes = list(executor.map(make_hash, *iterables: values, chunksize=1000))
15
16     end = time.perf_counter()
17
18     print("Broj hash vrijednosti:", len(hashes))
19     print("Paralelno vrijeme:", end - start)

```

Run zadatak-09-paralelno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-09-paralelno.py

Broj hash vrijednosti: 400000

Paralelno vrijeme: 0.5047240999992937

Slika 18 - Generisanje SHA-256 hash vrijednosti - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.3845533999847248}{0.5047240999992937} \approx \mathbf{0.761908}$$

Broj stringova	Dužina ulaznog bloka	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
400.000	1.000	0.385	0.505	0.762

Na skupu od 400.000 kratkih stringova, sekvencijalna verzija ostvarila je bolje vrijeme izvršavanja u odnosu na paralelnu verziju. Sekvencijalno hashiranje trajalo je 0,385 sekundi, dok je paralelnoj verziji bilo potrebno 0,505 sekundi. Na osnovu ovih rezultata, ostvareno ubrzanje iznosi 0,762, što predstavlja usporavanje od oko 24%.

Iako je izračunavanje SHA-256 hash vrijednosti CPU-bound operacija, u ovom slučaju su ulazni podaci veoma kratki, pa optimizovana implementacija algoritma u C-u može završiti obradu brže nego što procesi mogu međusobno razmijeniti podatke. Zbog toga paralelni pristup ne donosi očekivano ubrzanje. Pozitivni efekti paralelizacije mogu se očekivati pri obradi znatno većih količina podataka, kao što su velike datoteke, ili korištenjem višenitnog pristupa u slučajevima kada korištene C-ekstenzije oslobađaju GIL.

3.10. Primjer 10: Simulacija senzorskih očitanja

Potrebno je simulirati očitavanje podataka sa 6 nezavisnih senzora. Čitanje sa svakog pojedinačnog senzora traje 1 sekundu, pri čemu `time.sleep(1)` simulira mrežne/I-O zastoje i vrijeme odziva opreme. Funkcija zatim generiše nasumičnu vrijednost u opsegu od 20,0 do 30,0. Sekvencijalna verzija očitava senzore jedan po jedan unutar glavne niti, dok paralelna verzija koristi `ThreadPoolExecutor(max_workers=6)` iz modula `concurrent.futures`. Na ovaj način se svakom senzoru dodjeljuje po jedna nit, čime se omogućava konkurentno očitavanje svih senzora.

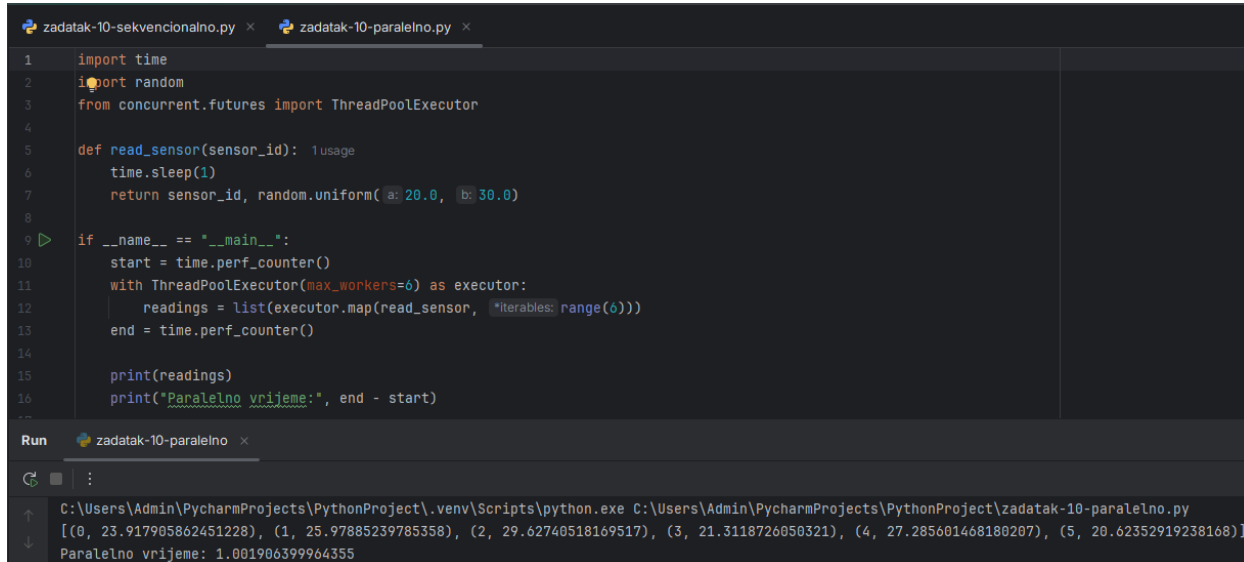


```
1 import time
2 import random
3
4 def read_sensor(sensor_id):
5     time.sleep(1)
6     return sensor_id, random.uniform(a=20.0, b=30.0)
7
8 start = time.perf_counter()
9 readings = [read_sensor(i) for i in range(6)]
10 end = time.perf_counter()
11
12 print(readings)
13 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-10-sekvencijalno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-10-sekvencijalno.py
[(0, 28.984633888641213), (1, 23.775812427471774), (2, 26.464914426321293), (3, 29.97270445469168), (4, 23.537917410379674), (5, 22.683509315712094)]
Sekvencijalno vrijeme: 6.001502699917182

Slika 19 - Simulacija senzorskih očitanja - sekvencijalno



```
1 import time
2 import random
3 from concurrent.futures import ThreadPoolExecutor
4
5 def read_sensor(sensor_id):
6     time.sleep(1)
7     return sensor_id, random.uniform(a=20.0, b=30.0)
8
9 if __name__ == "__main__":
10     start = time.perf_counter()
11     with ThreadPoolExecutor(max_workers=6) as executor:
12         readings = list(executor.map(read_sensor, range(6)))
13     end = time.perf_counter()
14
15     print(readings)
16     print("Paralelno vrijeme:", end - start)
```

Run zadatak-10-paralelno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-10-paralelno.py
[(0, 23.917905862451228), (1, 25.97885239785358), (2, 29.62740518169517), (3, 21.3118726050321), (4, 27.285601468180207), (5, 20.62352919238168)]
Paralelno vrijeme: 1.001906399964355

Slika 20 - Simulacija senzorskih očitanja - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{6.001502699917182}{1.001906399964355} \approx 5.990083$$

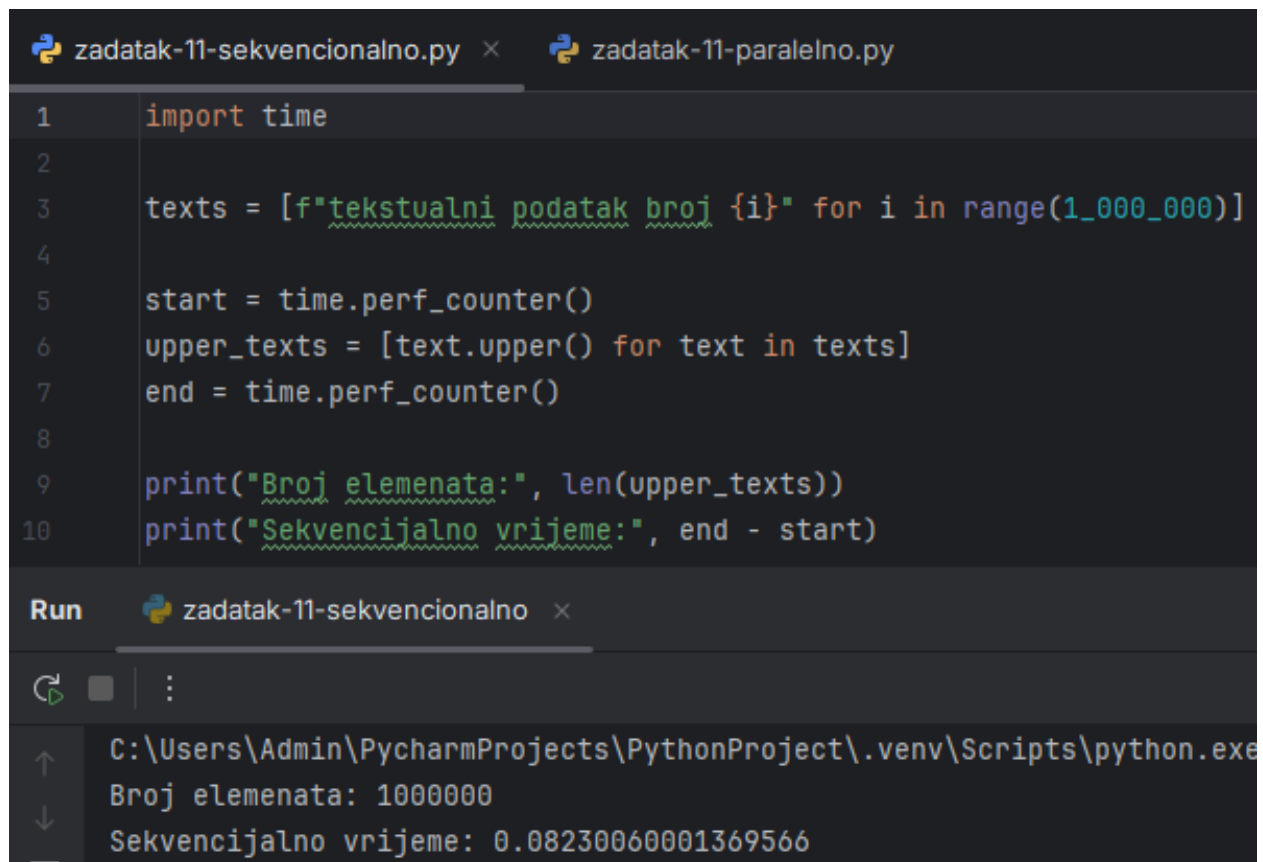
Broj senzora	Trajanje pojedinačnog očitanja	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
6	1.0 s	6.0015	1.0019	5.990

Na ovom primjeru paralelna verzija zasnovana na nitima ostvarila je skoro idealno linearno ubrzanje od približno $\times 6$. Sekvencijalno prikupljanje podataka sa 6 senzora trajalo je 6,0015 sekundi, dok je paralelna akvizicija pomoću radnih niti završena za svega 1,0019 sekundi, što daje ostvareno ubrzanje od 5,990.

Upotreba niti putem `ThreadPoolExecutor` predstavlja optimalno rješenje za mrežne i I/O operacije, kao što su rad sa senzorima, slanje HTTP zahtjeva ili rad sa bazama podataka. Budući da niti tokom čekanja na I/O operacije ne blokiraju izvršavanje drugih niti, više senzora može biti očitano istovremeno. Kada se koristi odgovarajući broj niti u odnosu na broj senzora ili uređaja, ukupno vrijeme izvršavanja približava se vremenu odziva najsporijeg pojedinačnog senzora.

3.11. Primjer 11: Pretvaranje velikog broja tekstova u velika slova

Potrebno je konvertovati 1.000.000 tekstualnih elemenata, odnosno stringova u formatu tekstualni podatak broj i, u velika slova. Sekvencijalna verzija koristi standardno razumijevanje listi (list comprehension) i ugrađenu metodu `text.upper()`. Paralelna verzija dijeli podatke na blokove od po 1.000 elemenata (`chunksize=1000`) i distribuira ih radnim procesima pomoću `ProcessPoolExecutor()`.



```
1 import time
2
3 texts = [f"tekstualni podatak broj {i}" for i in range(1_000_000)]
4
5 start = time.perf_counter()
6 upper_texts = [text.upper() for text in texts]
7 end = time.perf_counter()
8
9 print("Broj elemenata:", len(upper_texts))
10 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-11-sekvencionalno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj elemenata: 1000000
Sekvencijalno vrijeme: 0.08230060001369566

Slika 21 - Pretvaranje velikog broja tekstova u velika slova - sekvencionalno

```

1 import time
2 from concurrent.futures import ProcessPoolExecutor
3
4 def to_upper(text):
5     return text.upper()
6
7 if __name__ == "__main__":
8     texts = [f"tekstualni podatak broj {i}" for i in range(1_000_000)]
9
10    start = time.perf_counter()
11
12    with ProcessPoolExecutor() as executor:
13        upper_texts = list(executor.map(to_upper, texts, chunksize=1000))
14
15    end = time.perf_counter()
16
17    print("Broj elemenata:", len(upper_texts))
18    print("Paralelno vrijeme:", end - start)

```

Run zadatak-11-paralelno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmP
 Broj elemenata: 1000000
 Paralelno vrijeme: 0.8727903999388218

Slika 22 - Pretvaranje velikog broja tekstova u velika slova - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.08230060001369566}{0.8727903999388218} \approx \mathbf{0.094296}$$

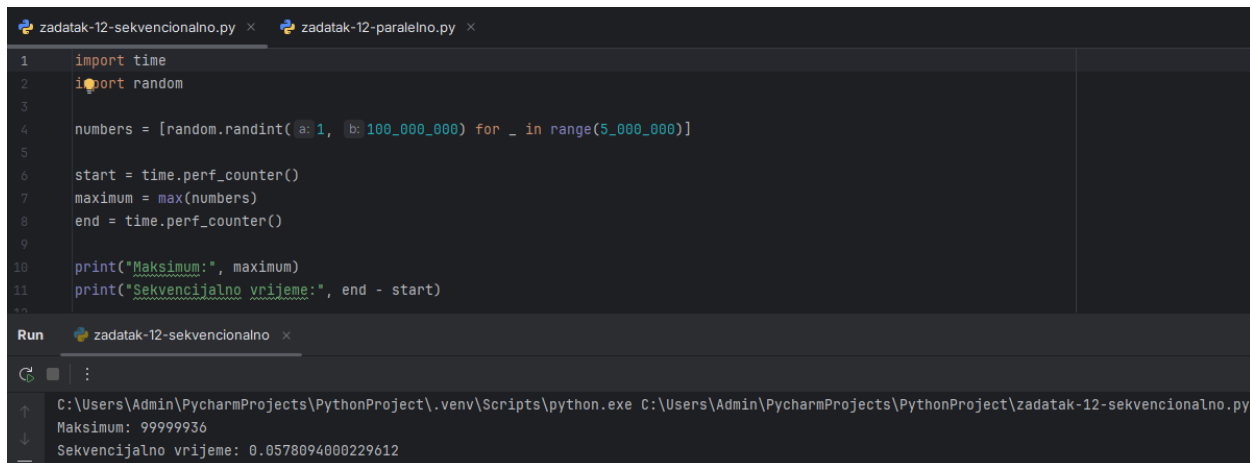
Ukupan broj elemenata	Dužina ulaznog bloka	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
1.000.000	1.000	0.082	0.873	0.094

Na skupu od 1.000.000 stringova, sekvencijalna verzija ostvarila je drastično bolje vrijeme izvršavanja. Sekvencijalna konverzija trajala je svega 0,082 sekunde, dok je paralelnoj verziji bilo potrebno 0,873 sekunde, što daje ubrzanje od 0,094, odnosno usporavanje veće od 10 puta.

Paralelizacija pomoću ProcessPoolExecutor nije efikasna za trivijalne string operacije kao što su upper(), lower() ili jednostavne zamjene karaktera. Sekvencijalna obrada unutar Python-a, kao i korištenje radnih niti ili biblioteka poput Polars i Pandas sa vektorskim operacijama, može biti znatno brže jer se izbjegavaju troškovi serijalizacije i prenosa podataka između zasebnih procesa.

3.12. Primjer 12: Traženje maksimalne vrijednosti u dijelovima liste

Potrebno je pronaći maksimalnu vrijednost u listi od 5.000.000 nasumično generisanih cijelih brojeva, pri čemu se vrijednosti nalaze u opsegu od 1 do 100.000.000. Sekvencijalna verzija koristi ugrađenu funkciju `max(numbers)` nad čitavom listom unutar glavnog procesa. Paralelna verzija dijeli listu na blokove (chunks) u skladu sa brojem raspoloživih CPU jezgara, koji se određuje pomoću `cpu_count()`. Za svaki blok se pomoću `ProcessPoolExecutor()` izračunava parcijalni maksimum, nakon čega se konačni maksimum pronalazi primjenom funkcije `max(partial_maximums)` nad dobijenim parcijalnim rezultatima.

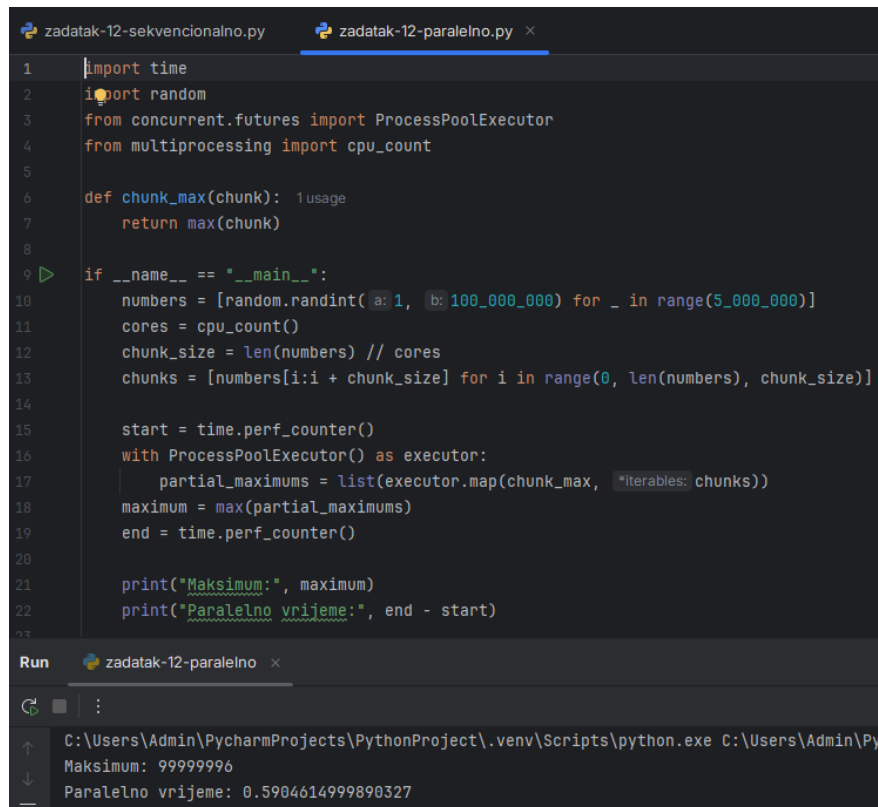


```
1 import time
2 import random
3
4 numbers = [random.randint(a=1, b=100_000_000) for _ in range(5_000_000)]
5
6 start = time.perf_counter()
7 maximum = max(numbers)
8 end = time.perf_counter()
9
10 print("Maksimum:", maximum)
11 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-12-sekvencionalno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-12-sekvencionalno.py
Maksimum: 99999936
Sekvencijalno vrijeme: 0.0578094000229612

Slika 23 - Traženje maksimalne vrijednosti u dijelovima liste - sekvencionalno



```
1 import time
2 import random
3 from concurrent.futures import ProcessPoolExecutor
4 from multiprocessing import cpu_count
5
6 def chunk_max(chunk): 1 usage
7     return max(chunk)
8
9 if __name__ == "__main__":
10     numbers = [random.randint(a=1, b=100_000_000) for _ in range(5_000_000)]
11     cores = cpu_count()
12     chunk_size = len(numbers) // cores
13     chunks = [numbers[i:i + chunk_size] for i in range(0, len(numbers), chunk_size)]
14
15     start = time.perf_counter()
16     with ProcessPoolExecutor() as executor:
17         partial_maximums = list(executor.map(chunk_max, *iterables: chunks))
18     maximum = max(partial_maximums)
19     end = time.perf_counter()
20
21     print("Maksimum:", maximum)
22     print("Paralelno vrijeme:", end - start)
```

Run zadatak-12-paralelno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\Py
Maksimum: 99999996
Paralelno vrijeme: 0.5904614999890327

Slika 24 - Traženje maksimalne vrijednosti u dijelovima liste - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.0578094000229612}{0.5904614999890327} \approx \mathbf{0.097905}$$

Ukupan broj elemenata	Pronađena maksimalna vrijednost	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
5.000.000	99.999.936	0.058	0.590	0.098

Na skupu od 5.000.000 elemenata, sekvencijalna verzija ostvarila je znatno bolje vrijeme izvršavanja. Sekvencijalna pretraga trajala je svega 0,058 sekundi, dok je paralelnoj verziji bilo potrebno 0,590 sekundi, što daje ubrzanje od 0,098, odnosno usporavanje veće od 10 puta.

Pronalaženje ekstremnih vrijednosti (min/max) nad običnim Python listama predstavlja računski veoma jednostavan zadatak i ne opravdava troškove kreiranja i komunikacije između više procesa. Za efikasniju paralelnu obradu ovakvih podataka u praksi se mogu koristiti mehanizmi poput dijeljene memorije (shared memory) ili biblioteke kao što je NumPy, koja funkcijom `numpy.max()` omogućava efikasnu obradu podataka smještenih u kontinualnom memorijskom nizu, uz znatno manje troškove u odnosu na prenos i serijalizaciju podataka između procesa.

3.13. Primjer 13: Simulacija Monte Carlo metode

Potrebno je procijeniti vrijednost broja π primjenom Monte Carlo simulacije nad ukupno 2.500.000 nasumično generisanih tačaka. Algoritam generiše koordinate x_i yiz intervala $[0, 1]$ i provjerava da li tačka pripada četvrtini kruga jediničnog poluprečnika, odnosno da li je ispunjen uslov $x^2 + y^2 \leq 1$. Sekvencijalna verzija izvršava svih 2.500.000 iteracija unutar jedne niti. Paralelna verzija dijeli ukupan broj tačaka na jednake dijelove u skladu sa brojem dostupnih procesorskih jezgara, pri čemu se broj tačaka po radniku određuje izrazom `points_per_worker = total_points // cores`. Dobijeni zadaci se zatim izvršavaju konkurentno pomoću `ProcessPoolExecutor()`.

```
zadatak-13-sekvencionalno.py × zadatak-13-paralelno.py

1 import time
2 import random
3
4 def simulate(points): 1 usage
5     inside = 0
6     for _ in range(points):
7         x = random.random()
8         y = random.random()
9         if x * x + y * y <= 1:
10             inside += 1
11     return inside
12
13 points = 2_500_000
14
15 start = time.perf_counter()
16 inside = simulate(points)
17 pi_estimate = 4 * inside / points
18 end = time.perf_counter()
19
20 print("Procjena broja pi:", pi_estimate)
21 print("Sekvencijalno vrijeme:", end - start)

Run zadatak-13-sekvencionalno ×

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Sc
Procjena broja pi: 3.1431632
Sekvencijalno vrijeme: 0.4033582000993192
```

Slika 25 - Simulacija Monte Carlo metode - sekvencionalno

```

1  import time
2  import random
3  from concurrent.futures import ProcessPoolExecutor
4  from multiprocessing import cpu_count
5
6  def simulate(points):
7      inside = 0
8      for _ in range(points):
9          x = random.random()
10         y = random.random()
11         if x * x + y * y <= 1:
12             inside += 1
13     return inside
14
15  if __name__ == "__main__":
16      total_points = 2_500_000
17      cores = cpu_count()
18      points_per_worker = total_points // cores
19
20      start = time.perf_counter()
21      with ProcessPoolExecutor() as executor:
22          results = list(executor.map(simulate, *iterables: [points_per_worker] * cores))
23      inside = sum(results)
24      pi_estimate = 4 * inside / total_points
25      end = time.perf_counter()
26
27      print("Procjena broja pi:", pi_estimate)
28      print("Paralelno vrijeme:", end - start)

```

Run zadatak-13-paralelno

```

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\Py
Procjena broja pi: 3.142632
Paralelno vrijeme: 0.35771759995259345

```

Slika 26 - Simulacija Monte Carlo metode - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.4033582000993192}{0.35771759995259345} \approx \mathbf{1.127588}$$

Ukupan tačka	broj	Procjena vrijednosti π	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
2.500.000		3.142632	0.403	0.358	1.127

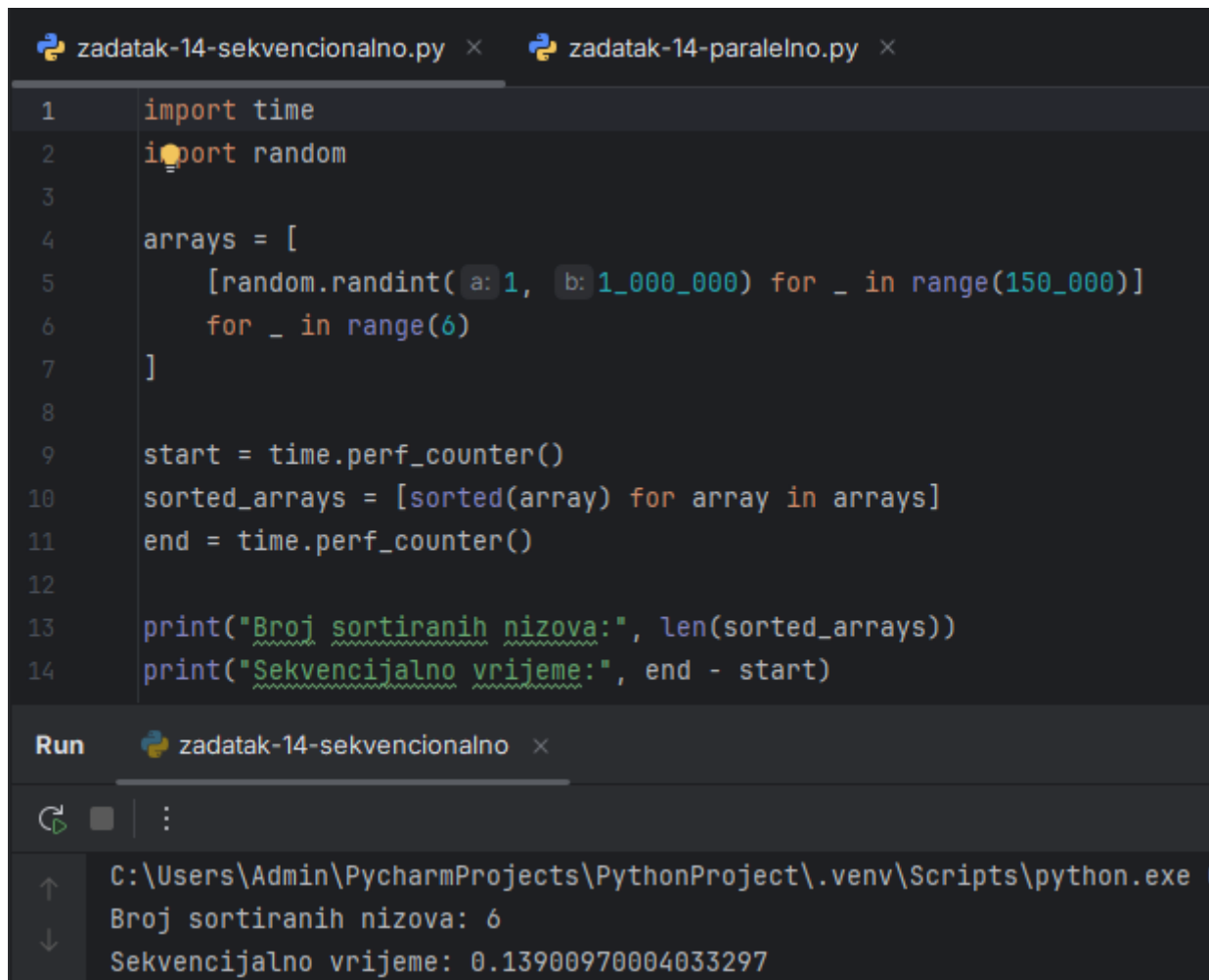
Na skupu od 2.500.000 tačaka, paralelna verzija ostvarila je ubrzanje od $1,13\times$ u odnosu na sekvencijalno izvršavanje. Sekvencijalna simulacija trajala je 0,403 sekunde, dok je paralelnoj verziji bilo potrebno 0,358 sekundi.

Monte Carlo simulacija predstavlja pogodan kandidat za paralelizaciju jer se pojedinačne iteracije mogu izvršavati nezavisno jedna od druge. Povećanjem broja tačaka na 10.000.000 ili više, fiksni

trošak pokretanja i koordinacije procesa imao bi manji uticaj na ukupno vrijeme izvršavanja, pa bi se ostvareno ubrzanje moglo dodatno približiti teorijskom maksimumu u odnosu na broj fizičkih jezgara procesora.

3.14. Primjer 14: Sortiranje nezavisnih nizova

Potrebno je sortirati 6 nezavisnih nizova, pri čemu svaki niz sadrži po 150.000 nasumično generisanih cijelih brojeva u opsegu od 1 do 1.000.000. Sekvencijalna verzija sortira nizove jedan po jedan unutar glavnog procesa, koristeći ugrađenu funkciju `sorted()`. Paralelna verzija raspoređuje sortiranje svakog pojedinačnog niza na radne procese iz bazena `ProcessPoolExecutor()`, čime se omogućava istovremena obrada svih nizova.



```
1 import time
2 import random
3
4 arrays = [
5     [random.randint(a: 1, b: 1_000_000) for _ in range(150_000)]
6     for _ in range(6)
7 ]
8
9 start = time.perf_counter()
10 sorted_arrays = [sorted(array) for array in arrays]
11 end = time.perf_counter()
12
13 print("Broj sortiranih nizova:", len(sorted_arrays))
14 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-14-sekvencionalno

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj sortiranih nizova: 6
Sekvencijalno vrijeme: 0.13900970004033297

Slika 27 - Sortiranje nezavisnih nizova - sekvencionalno

```

1 import time
2 import random
3 from concurrent.futures import ProcessPoolExecutor
4
5 def sort_array(array):
6     return sorted(array)
7
8 if __name__ == "__main__":
9     arrays = [
10         [random.randint(a: 1, b: 1_000_000) for _ in range(150_000)]
11         for _ in range(6)
12     ]
13
14     start = time.perf_counter()
15     with ProcessPoolExecutor() as executor:
16         sorted_arrays = list(executor.map(sort_array, *iterables: arrays))
17     end = time.perf_counter()
18
19     print("Broj sortiranih nizova:", len(sorted_arrays))
20     print("Paralelno vrijeme:", end - start)

```

Run zadatak-14-paralelno

```

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\U
Broj sortiranih nizova: 6
Paralelno vrijeme: 0.2765437999041751

```

Slika 28 - Sortiranje nezavisnih nizova - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.13900970004033297}{0.2765437999041751} \approx \mathbf{0.502668}$$

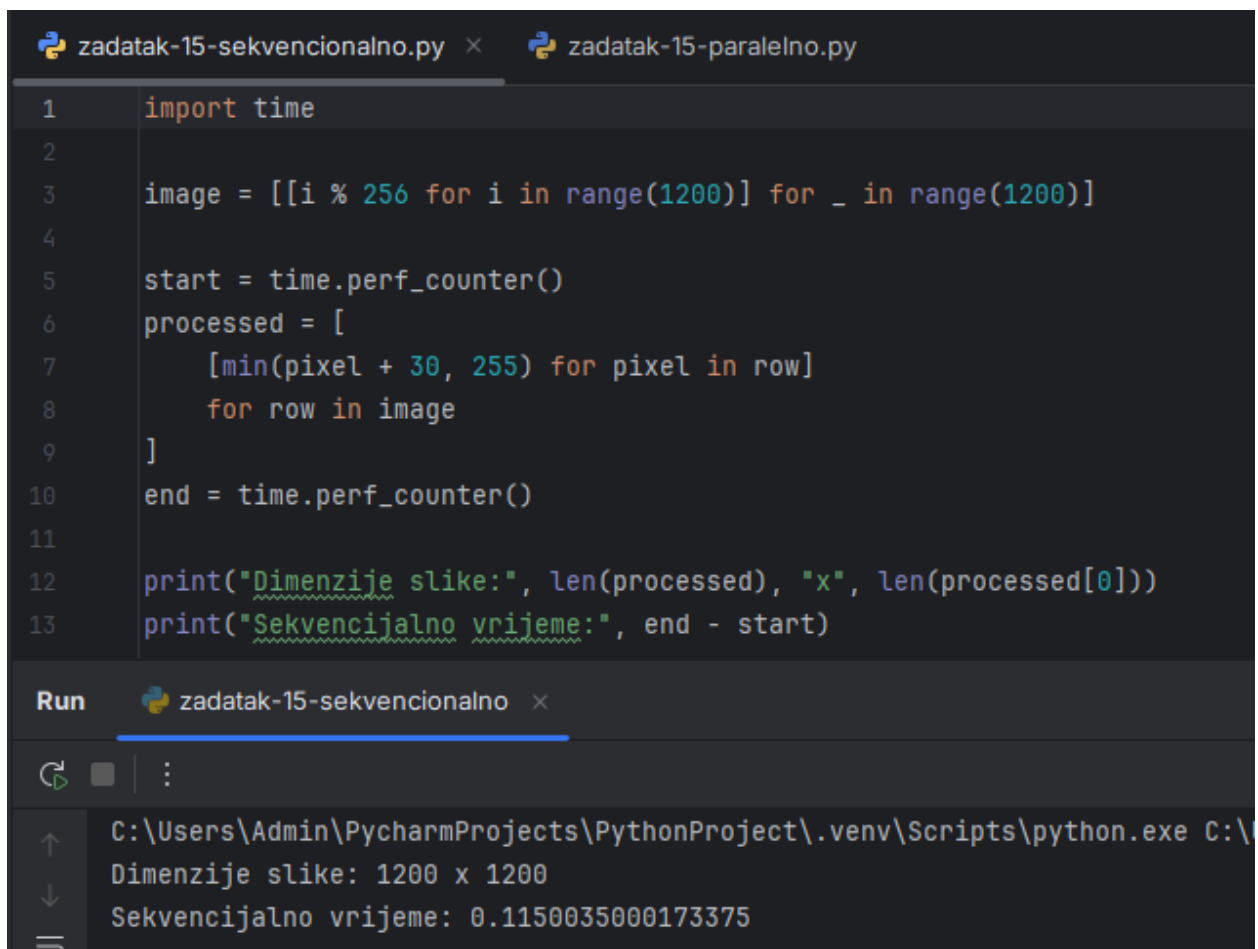
Broj nizova	Veličina pojedinačnog niza	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
6	150.000	0.139	0.277	0.503

Na primjeru 6 nizova sa po 150.000 elemenata, sekvencijalna verzija ostvarila je duplo bolje vrijeme izvršavanja. Sekvencijalno sortiranje trajalo je 0,139 sekundi, dok je paralelnoj verziji bilo potrebno 0,277 sekundi, što daje ostvareno ubrzanje od 0,503, odnosno usporavanje od oko 2×.

Paralelizacija sortiranja nezavisnih nizova pomoću `ProcessPoolExecutor` postaje isplativa tek kada je veličina pojedinačnih nizova znatno veća, na primjer nekoliko miliona elemenata po nizu, ili kada se koristi dijeljena memorija, poput `multiprocessing.shared_memory` ili `NumPy` nizova. Na taj način može se smanjiti potreba za serijalizacijom i prenosom velikih količina podataka između procesa.

3.15. Primjer 15: Obrada jednostavne slike kao matrice

Potrebno je izvršiti posvjetljivanje slike dimenzija 1200×1200 piksela, odnosno ukupno 1.440.000 piksela predstavljenih u obliku matrice. Svakom pikselu dodaje se vrijednost 30, pri čemu maksimalna vrijednost ne smije preći 255, što se postiže izrazom `min(pixel + 30, 255)`. Sekvencijalna verzija obrađuje matricu red po red pomoću dvostruke list comprehension petlje unutar glavnog procesa. Paralelna verzija mapira obradu pojedinačnih redova pomoću funkcije `process_row` na radne procese iz bazena `ProcessPoolExecutor()`.



```
zadatak-15-sekvencionalno.py x zadatak-15-paralelno.py
1 import time
2
3 image = [[i % 256 for i in range(1200)] for _ in range(1200)]
4
5 start = time.perf_counter()
6 processed = [
7     [min(pixel + 30, 255) for pixel in row]
8     for row in image
9 ]
10 end = time.perf_counter()
11
12 print("Dimenzije slike:", len(processed), "x", len(processed[0]))
13 print("Sekvencijalno vrijeme:", end - start)

Run zadatak-15-sekvencionalno x
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\
Dimenzije slike: 1200 x 1200
Sekvencijalno vrijeme: 0.1150035000173375
```

Slika 29 - Obrada jednostavne slike kao matrice - sekvencionalno

```

1  import time
2  from concurrent.futures import ProcessPoolExecutor
3
4  def process_row(row):
5      return [min(pixel + 30, 255) for pixel in row]
6
7  if __name__ == "__main__":
8      image = [[i % 256 for i in range(1200)] for _ in range(1200)]
9
10     start = time.perf_counter()
11     with ProcessPoolExecutor() as executor:
12         processed = list(executor.map(process_row, image))
13     end = time.perf_counter()
14
15     print("Dimenzije slike:", len(processed), "x", len(processed[0]))
16     print("Paralelno vrijeme:", end - start)

```

Run zadatak-15-paralelno

```

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:
Dimenzije slike: 1200 x 1200
Paralelno vrijeme: 0.5078541999682784

```

Slika 30 - Obrada jednostavne slike kao matrice - paralelno

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.1150035000173375}{0.5078541999682784} \approx \mathbf{0.226450}$$

Dimenzije slike	Ukupno piksela	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
1200 x 1200	1.440.000	0.115	0.508	0.226

Za sliku dimenzija 1200×1200 piksela, sekvencijalna verzija ostvarila je znatno bolje vrijeme izvršavanja. Sekvencijalna obrada trajala je 0,115 sekundi, dok je paralelnoj verziji bilo potrebno 0,508 sekundi, što daje ostvareno ubrzanje od 0,226, odnosno usporavanje od oko 4,4×.

Obične Python matrice i liste nisu najpogodnije za elementarne računarske operacije nad slikama u multiprocesiranom okruženju. Za efikasnu obradu slika u praksi se koriste namjenske biblioteke poput NumPy i OpenCV, koje rade sa kontinualnim blokovima memorije i omogućavaju efikasnije izvršavanje vektorskih operacija. Na taj način se smanjuje komunikacioni trošak i potreba za serijalizacijom podataka, dok se dio paralelizacije može izvršavati interno, bez potrebe za ručnim raspoređivanjem pojedinačnih redova na zasebne procese.

4. PREGLED REZULTATA

U nastavku je prikazan zbirni pregled svih 15 testiranih primjera, sa podacima o korištenom načinu paralelizacije, veličini ulaznih podataka, izmjerenom vremenu izvršavanja i ostvarenom ubrzanju.

Br.	Naziv primjera	Vrsta paralelizacije	Veličina ulaza	Sekvencijalno (s)	Paralelno (s)	Ubrzanje	Bolja verzija
1.	Suma kvadrata brojeva	Procesi	5.000.000	0,4	0,694	0,577	Sekvencijalna
2.	Obrada liste ocjena	Procesi	3.000.000	0,113	0,389	0,29	Sekvencijalna
3.	Provjera djeljivosti	Procesi	8.000.000	0,416	0,942	0,442	Sekvencijalna
4.	Simulacija sporih operacija	Niti	5 operacija	10,002	2,002	4,996	Paralelna
5.	Analiza tekstualnih dokumenata	Procesi	3 dokumenta	0,312	0,395	0,788	Sekvencijalna
6.	Računanje prostih brojeva	Procesi	30.000	0,072	8	0,009	Sekvencijalna
7.	Obrada redova matrice	Procesi	3000×1000	0,021	0,834	0,026	Sekvencijalna
8.	Filtriranje podataka	Procesi	12.000.000	0,459	1,635	0,281	Sekvencijalna
9.	Generisanje SHA-256 hash-a	Procesi	400.000	0,385	0,505	0,762	Sekvencijalna
10.	Simulacija senzorskih očitavanja	Niti	6 senzora	6,0015	1,0019	5,99	Paralelna
11.	Konverzija teksta u velika slova	Procesi	1.000.000	0,082	0,873	0,094	Sekvencijalna
12.	Traženje maksimalne vrijednosti	Procesi	5.000.000	0,058	0,59	0,098	Sekvencijalna
13.	Monte Carlo simulacija	Procesi	2.500.000	0,403	0,358	1,127	Paralelna

14.	Sortiranje nezavisnih nizova	Procesi	6 150.000 ×	0,139	0,277	0,503	Sekvencijalna
15.	Obrada slike kao matrice	Procesi	1200×1200	0,115	0,508	0,226	Sekvencijalna

ZAKLJUČAK

Na osnovu sprovedene analize može se zaključiti da paralelno izvršavanje programa u Pythonu ne predstavlja univerzalno rješenje za ubrzavanje svih vrsta zadataka. Rezultati 15 testiranih primjera pokazali su da efikasnost paralelizacije u velikoj mjeri zavisi od prirode problema, količine podataka i odnosa između vremena potrebnog za obradu i troškova pokretanja i komunikacije između procesa ili niti.

Najizraženije poboljšanje ostvareno je kod zadataka koji uključuju čekanje na spoljne resurse. U simulaciji sporih operacija paralelna verzija ostvarila je ubrzanje od $4,996\times$, dok je kod simulacije senzorskih očitavanja ostvareno ubrzanje od $5,990\times$. Ovi rezultati potvrđuju da su niti, odnosno ThreadPoolExecutor, veoma pogodne za I/O-bound zadatke kod kojih program značajan dio vremena provodi čekajući odgovor mreže, senzora, baze podataka ili drugog eksternog resursa.

Kod CPU-bound zadataka rezultati su bili različiti. Monte Carlo simulacija ostvarila je ubrzanje od $1,127\times$, što pokazuje da procesi mogu donijeti određeno poboljšanje kada se računski posao može ravnomjerno podijeliti između više jezgara. S druge strane, kod računanja prostih brojeva paralelna verzija bila je više od 110 puta sporija jer su pojedinačni zadaci prosljeđivani procesima bez odgovarajućeg grupisanja. Ovaj primjer jasno pokazuje koliko način raspodjele posla može uticati na konačne performanse.

U većem broju ostalih primjera sekvencijalna verzija bila je brža. To se posebno vidi kod jednostavnih operacija nad velikim Python listama, kao što su filtriranje, pronalaženje maksimuma, pretvaranje stringova u velika slova, sabiranje elemenata matrice i sortiranje. U takvim slučajevima vrijeme potrebno za samu obradu pojedinačnog elementa bilo je premalo da bi opravdalo dodatne troškove kreiranja procesa, serijalizacije i prenosa podataka. Zbog toga je paralelna verzija često ostvarila lošije rezultate uprkos korištenju više procesorskih jezgara.

Analiza je takođe pokazala značaj pravilnog izbora struktura podataka i biblioteka. Kod velikih numeričkih skupova podataka efikasnije rješenje mogu predstavljati biblioteke poput NumPy, koje koriste optimizovane operacije nad kontinualnim memorijskim blokovima. Na taj način moguće je smanjiti troškove karakteristične za obradu običnih Python lista i ostvariti bolje performanse bez potrebe za ručnim raspoređivanjem velikog broja malih zadataka.

Konačno, može se zaključiti da odluka o korištenju sekvencijalnog ili paralelnog pristupa treba biti zasnovana na konkretnom problemu i rezultatima mjerenja. Paralelizacija ima najveći smisao kada su zadaci dovoljno veliki, međusobno nezavisni i kada vrijeme njihove obrade opravdava dodatni trošak komunikacije i koordinacije. U suprotnom, jednostavan sekvencijalni program može biti brži, pregledniji i efikasniji. Zbog toga je prije primjene paralelnog programiranja potrebno analizirati karakteristike zadatka, izabrati odgovarajući model izvršavanja i eksperimentalno provjeriti da li paralelizacija zaista donosi očekivano ubrzanje.

POPIS SLIKA

Slika 1 - Suma kvadrata brojeva - sekvencijalno.....	8
Slika 2 - Suma kvadrata brojeva - paralelno	9
Slika 3 - Obrada liste ocjena - sekvencionalno	10
Slika 4 - Obrada liste ocjena - paralelno	11
Slika 5 - Provjera djeljivosti velikog broja elemenata - sekvencionalno.....	12
Slika 6 - Provjera djeljivosti velikog broja elemenata - paralelno.....	13
Slika 7 - Simulacija više sporih operacija - sekvencionalno.....	14
Slika 8 - Simulacija više sporih operacija - paralelno.....	14
Slika 9 - Analiza više tekstualnih dokumenata - sekvencionalno	15
Slika 10 - Analiza više tekstualnih dokumenata - paralelno	16
Slika 11 - Računanje prostih brojeva - sekvencijalno.....	17
Slika 12 - Računanje prostih brojeva - paralelno.....	18
Slika 13 - Paralelna obrada redova matrice - sekvencionalno	19
Slika 14 - Paralelna obrada redova matrice - paralelno	19
Slika 15 - Filtriranje podataka iz velike liste - sekvencionalno	20
Slika 16 - Filtriranje podataka iz velike liste - paralelno	21
Slika 17 - Generisanje SHA-256 hash vrijednosti - sekvencionalno.....	22
Slika 18 - Generisanje SHA-256 hash vrijednosti - paralelno.....	23
Slika 19 - Simulacija senzorskih očitavanja - sekvencionalno	24
Slika 20 - Simulacija senzorskih očitavanja - paralelno	24
Slika 21 - Pretvaranje velikog broja tekstova u velika slova - sekvencionalno.....	25
Slika 22 - Pretvaranje velikog broja tekstova u velika slova - paralelno.....	26
Slika 23 - Traženje maksimalne vrijednosti u dijelovima liste - sekvencionalno.....	27
Slika 24 - Traženje maksimalne vrijednosti u dijelovima liste - paralelno.....	27
Slika 25 - Simulacija Monte Carlo metode - sekvencionalno.....	29
Slika 26 - Simulacija Monte Carlo metode - paralelno.....	30
Slika 27 - Sortiranje nezavisnih nizova - sekvencionalno	31
Slika 28 - Sortiranje nezavisnih nizova - paralelno	32
Slika 29 - Obrada jednostavne slike kao matrice - sekvencionalno.....	33
Slika 30 - Obrada jednostavne slike kao matrice - paralelno.....	34

LITERATURA

- [1] Python Software Foundation, »concurrent.futures — Launching parallel tasks,« Python 3 Documentation, 2026. [Mrežno]. Available: <https://docs.python.org/3/library/concurrent.futures.html>.
- [2] Python Software Foundation, »multiprocessing — Process-based parallelism,« Python 3 Documentation, 2026. [Mrežno]. Available: <https://docs.python.org/3/library/multiprocessing.html>.
- [3] Python Software Foundation, »threading — Thread-based parallelism,« Python 3 Documentation, 2026. [Mrežno]. Available: <https://docs.python.org/3/library/threading.html>.
- [4] L. Pozo Ramos, »Global Interpreter Lock (GIL),« Real Python, 2026. [Mrežno]. Available: <https://realpython.com/ref/glossary/gil/>.